



基于多目标跟踪 (MOT) 的自动驾驶激光 SLAM 系统

ME5400 项目

Wu Zining (A0294373W)

2026 年 4 月 14 日

目录

1 引言	1
2 研究现状分析	2
2.1 动态场景 SLAM: 从动态点剔除到保守降权	3
2.2 对象感知联合估计: 让动态信息更深地进入定位	3
2.3 对比分析与存在的差距	4
3 整体系统架构与定位增强数据流	5
3.1 FAST-LIO 前端	6
3.2 PointPillars 对象观测模块	7
3.3 MCTrack 对象状态估计模块与数据流细节	8
3.4 时间对齐、坐标系与失效模式	10
4 工程实现与系统集成	10
4.1 整体代码组织	10
4.2 在线系统设计	11
4.3 结果归档与脚本化实验管线	13
4.4 ROS 话题、消息接口与集成困难	14
5 动态权重优化	14
5.1 设计动机与代码位置	14
5.2 权重计算	15
5.3 从跟踪对象到点级权重	16
5.4 边界情况与参数敏感性	16
6 联合后端优化	17
6.1 仅自车滑窗问题的提出	17
6.2 残差建模与门控策略	18
6.3 滑窗更新逻辑与实际行为	19
6.4 联合后端的价值与局限	19
7 实验分析	20
7.1 实验设置与结果来源	20

目录	2
7.2 在线实验：汇总结果	21
7.3 综合分析与失效模式	24
7.4 从现象到解释	25
7.5 实验公平性与报告纪律	25
8 结论与未来计划	25
参考文献	27
致谢	28
附录：在线实验全量图片	29

摘要

本报告围绕“如何在动态物体干扰下提升激光 SLAM 的定位精度”这一问题展开。动态车辆、行人等交通参与者会破坏 SLAM 所依赖的静态世界假设，进而引发地图污染、配准退化和轨迹漂移。为此，本项目以 FAST-LIO 为主定位骨干，并将 PointPillars 与 MCTrack 组织为对象级动态信息模块：前者提供带时间戳的 3D 检测，后者生成可跨帧传播的对象状态。这些对象信息首先用于前端的软动态降权 (soft down-weighting)，以降低动态点对扫描匹配的误导；同时，它们也被送入仅包含自车位姿 (ego-only) 的探索性滑窗后端，用于测试对象观测能否进一步提供额外约束。

所有定量讨论均基于 Results/ 目录下 7 个所选 KITTI Tracking 序列 (0003, 0006, 0007, 0009, 0010, 0013, 0020) 的归档在线实验结果。这些序列覆盖了不同的道路类型、动态强度与漂移条件，因此在当前仓库条件下构成了有意义的 KITTI 多序列“多场景”验证，而非跨数据集泛化。报告强调算法设计意图与工程实现细节，包括 ROS 接口、时间同步、位姿缓冲、标准结果归档以及失效模式分析。在这 7 个所选在线序列上，前端配置 FAST-LIO+MCTrack 在 ATE RMSE 指标上均优于纯 FAST-LIO 基线，表明目前最稳定、证据最充分的收益来自对象感知的前端动态降权。探索性联合后端虽在部分漂移严重的序列上有所帮助，但其稳定性较弱，在速度外推、时间戳对齐或目标门控不可靠时可能失效，序列 0009 便是最典型的例子。因此，本报告并不将项目表述为一个完整的 SLAMMOT 系统，而是把它定位为一个围绕动态场景定位精度提升而设计的可复现实验平台，用于研究对象级动态信息何时能够稳定帮助定位、何时又会引入新的不确定性。完整代码、脚本与结果归档入口公开于 <https://github.com/XC-CN/ME5400>。

关键词： FAST-LIO；动态场景 SLAM；定位精度提升；对象感知动态抑制；KITTI 多序列验证；ROS 重现性

1 引言

在动态道路场景中，激光 SLAM 默认依赖“环境中大多数几何结构是静态的”这一假设；然而，车辆、行人、自行车等交通参与者会持续破坏这一前提，使动态点被错误写入局部地图，并进一步诱发点云配准退化、姿态漂移累积以及跨场景稳定性下降。对于自动驾驶任务而言，这不是边缘问题，而是决定定位精度上限的核心矛盾。

本项目因此不把“构建一个完整的 SLAM+MOT 系统”作为首要目标，而是把检测与跟踪模块视为服务定位的对象级动态信息来源。系统采用 FAST-LIO [1] 作为主定位骨干，使用 PointPillars [2] 产生 3D 检测，并利用 MCTrack [3] 将离散检测整理为连续轨迹状态。随后，这些对象状态通过两种方式作用于定位：一是在 FAST-LIO 前端中转化为软动态权重，保守地抑制动态区域对扫描匹配的影响；二是送入仅含自车位姿 (ego-only) 的联合后端，作为探索性扩展来测试对象观测是否能在漂移主导场景下提供

额外约束。为平衡项目周期与工程鲁棒性，系统采用了软降权和松耦合消息闭环方案，而非强依赖紧耦合的完整对象图优化。

据此，本项目追求四个核心目标：**研究目标**为验证对象级动态信息能否在动态场景中稳定提升 LiDAR SLAM 定位精度；**工程目标**为构建可重复运行且组件边界清晰的标准 ROS 管线；**评估目标**为分析该收益在 KITTI 多序列场景中的稳定性、适用条件与失效边界；**诊断目标**则为识别探索性联合后端在速度外推、时间对齐和门控可靠性方面的主要脆弱点。

具体而言，本报告及实现的贡献可总结如下：

- 实现了以 FAST-LIO 为主定位骨干、以 PointPillars 和 MCTrack 为对象级动态观测模块的完整 ROS 集成，采用项目定义的标准消息接口。
- 构建了对象感知前端闭环：FAST-LIO 位姿稳定 MCTrack 的跨帧关联，而 MCTrack 输出被转化为 FAST-LIO 预处理中的点级软降权。
- 建立了脚本驱动的实验框架，支持基线与优化运行的一键切换、Results/ 下的标准归档以及统一的多序列定量评估。
- 通过失效感知分析表明，前端对象感知动态降权是当前最稳定的定位改进源，而探索性后端在速度外推及时间不确定性面前仍显脆弱。

选择 KITTI Tracking 作为验证数据集，正是因为其不同序列覆盖了不同的道路类型、动态强度和漂移条件（本报告使用 0003, 0006, 0007, 0009, 0010, 0013, 0020），能够构成当前仓库条件下有意义的“多场景”定位验证 [4]。**本报告并非旨在声称达到了 SOTA 性能**，也不把多目标跟踪本身作为独立研究终点；它记录的是一个围绕动态场景定位精度提升而设计的工程原型，用于展示对象级动态信息何时能够稳定帮助定位、何时又会引入新的不确定性。

2 研究现状分析

为了更准确地定位本项目，相关工作更适合沿着“动态信息如何作用于定位”这一主线来梳理，而不是把 SLAM 与 MOT 当作两个对等目标分别叙述。第一类工作直接面向**动态场景 SLAM**，通过剔除、分割、背景恢复或软降权来减少动态区域对位姿估计的污染。第二类工作进一步走向**对象感知联合估计**，尝试把对象状态、关联置信度或运动先验纳入定位与优化过程。本项目位于这两类方法之间：它已经不再把动态物体仅仅视为待删除的噪声，但也尚未形成完整的对象中心联合优化器。因此，一个有意义的对比应围绕动态信息是否直接服务定位、为此付出了多少系统复杂度、以及在实时场景下是否可落地来展开。

表 1: 与本文定位精度主线相关的代表性动态场景 SLAM 与对象感知联合估计方法比较

工作	传感器 / 前端	动态处理方式	实时性 / 复杂度	与本文关系
StaticFusion	RGB-D; 稠密直接法	联合分割与背景融合	面向实时稠密建图 [5]	说明保守保留背景结构和加权处理可以直接改善定位。
DS-SLAM	视觉; ORB-SLAM2 + 语义线程	语义过滤 + 运动一致性	多线程实时, 工程友好 [6]	说明松耦合工程系统也能提高动态场景定位鲁棒性。
DynaSLAM	视觉; ORB-SLAM2 家族	动态检测 + 背景恢复	较慢但精度高 [7]	代表把动态物体主要视为定位干扰源的强基线。
Conf SLAM-MOT	激光; 激光里程计	置信度引导关联与对象因子	紧耦合, 高复杂度 [8]	启发对象置信度不应只用于筛选, 也应影响定位约束强度。
LIMOT	激光 + 惯性; LIO	实时联合 LIO 与 MOT	实时, 但实现复杂 [9]	是本文长期目标最接近的紧耦合参考。
IMM-SLAMMOT	双目视觉; ORB-SLAM2	多运动模型切换	建模更强, 代价更高 [10]	说明单一匀速模型不足以稳定支撑动态场景定位。
Online Dynamic SLAM	双目视觉; 自研 VO	基于 iSAM2 的动态增量平滑	更可扩展的在线后端 [11]	指出动态对象一旦进入估计问题, 增量式后端就变得关键。
DL-SLOT	激光; 激光里程计	定位-跟踪协同图优化	紧耦合, 优化层共享信息 [12]	说明对象级约束可以在优化层直接服务定位。

2.1 动态场景 SLAM: 从动态点剔除到保守降权

StaticFusion、DS-SLAM 和 DynaSLAM 共同定义了动态场景 SLAM 的典型起点 [5, 6, 7]。尽管它们分别基于 RGB-D 或视觉前端, 而非本文采用的激光惯性里程计, 但它们确立了三个至今仍然有效的思想: 应显式检测动态区域; 应尽量保留真实背景结构而非盲目硬删除; 模块化系统即使不把所有内容并入同一个优化器, 也依然可能稳定改善定位鲁棒性。对本文而言, 这类工作最重要的意义不是“它们属于哪一种传感器路线”, 而是它们都证明了动态物体处理能够直接改变定位结果。

相对于这些方法, 本文的不同之处在于: 动态物体不再只被看作需要抑制的噪声区域。PointPillars 和 MCTrack 生成的对象级信息在过滤之后仍被保留下来, 从而允许系统在保守降权之外进一步探索更强的对象约束。这种设计继承了动态场景 SLAM 文献中“先稳住定位、再决定是否更深利用动态信息”的务实思路。

2.2 对象感知联合估计: 让动态信息更深地进入定位

Conf SLAMMOT、LIMOT、IMM-SLAMMOT、Online Dynamic SLAM 和 DL-SLOT 展示了比“动态点过滤”更激进的一条路线 [8, 9, 10, 11, 12]。这些工作的重要性不在于把 SLAM 与 MOT 拼成了一个更完整的系统名称, 而在于它们证明: 如果对象状态、关联置信度和运动先验能够进入估计问题本身, 动态信息就有机会从“待抑制的干扰”

转化为“有条件可利用的定位线索”。代价则是更高的状态维度、更严格的同步需求以及更复杂的因子设计。

其中，Conf SLAMMOT 强调优化内部的置信度引导关联 [8]，提醒我们对对象置信度不应只是保留或丢弃的阈值，而应影响约束强度；LIMOT 则展示了实时联合 LIO 与 MOT 的紧耦合滑窗形式 [9]，是本文长期目标最接近的参考。与这类系统相比，当前项目仍通过消息接口连接 FAST-LIO、PointPillars、MCTrack 和后端，但已经在模块边界暴露了带时间戳的位姿、对象观测、轨迹状态以及运动属性，为后续更强的联合优化打下了数据基础。

IMM-SLAMMOT、Online Dynamic SLAM 与 DL-SLOT 则分别从运动模型、后端形式和优化层集成三个角度补充了这一方向 [10, 11, 12]。IMM-SLAMMOT 说明交通参与者并不总能被单一匀速模型描述；这一点与本文在 Sequence 0009 中观察到的后端退化高度相关。Online Dynamic SLAM 强调，一旦动态对象进入估计问题，增量式平滑与窗口管理就会成为核心设计约束。DL-SLOT 进一步表明，对象级约束若要稳定发挥作用，最终往往需要在优化层而非仅在消息层共享信息。

2.3 对比分析与存在的差距

从上述工作中可以归纳出三点。首先，动态物体处理确实能够稳定改变定位结果，但最成熟、最稳妥的方法通常仍停留在过滤、分割、背景恢复或保守降权层面。其次，一旦希望让对象状态更强地参与定位约束，系统就会迅速走向更高复杂度的联合优化框架。第三，实时性、同步质量和置信度建模仍然是决定方法能否落地的关键，因为它们直接影响对象信息究竟是在帮助定位，还是把噪声重新注入位姿估计。

相对于这一研究格局，本项目占据了一个有意为之的中间位置。它已经利用对象级动态信息直接改善 FAST-LIO 前端，因此超越了纯粹的动态点剔除；但它仍然通过显式消息接口连接检测、跟踪与后端，而不是共享单一对象中心状态图。其价值不在于宣称已经做成了完整 SLAMMOT 系统，而在于验证了一个更贴近当前工程条件的问题：在不立即引入高复杂度联合优化的情况下，对象级动态信息能否稳定提升动态场景下的定位精度。当前结果表明，前端对象感知动态降权给出了肯定答案，而真正的差距则在于：系统已具备所需的信息流，却尚未具备足够强的状态表示、置信度建模与因子设计，去把这些信息稳定地提升为更强的后端定位约束。

3 整体系统架构与定位增强数据流

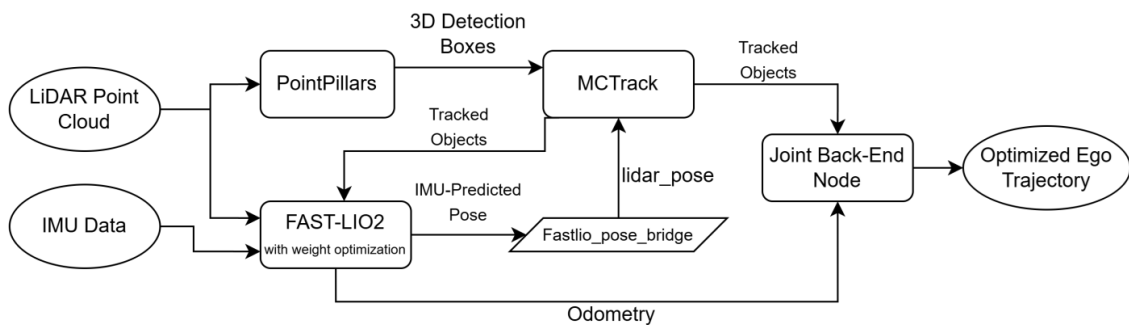


图 1: 整体系统架构。可以将其理解为“主定位骨干 + 对象观测模块 + 探索性后端”的三层数据流。

图 1 作为一个数据依赖图比作为静态方框图更容易理解。第一层是主定位骨干：FAST-LIO 负责从原始激光与 IMU 数据中输出自车位姿。第二层是对象级动态观测模块：PointPillars 从相同的激光雷达输入发布 3D 检测，fastlio_pose_bridge.py 将 FAST-LIO 里程计转换为 MCTrack 所需的激光位姿接口，而 MCTrack 再将位姿与检测对齐以形成连贯的目标轨迹。第三层是定位增强链路：跟踪到的对象一方面反馈给 FAST-LIO 用于动态点降权，另一方面送往 ego-only 后端，作为探索性机制测试对象观测是否能够进一步修正自车轨迹。

这种组织形式是必要的，因为各模块服务于不同层级的定位需求。FAST-LIO 处理原始几何与惯性传播，PointPillars 和 MCTrack 负责生成对象级动态观测，而后端只承担短时域的探索性自车修正。保持这些阶段显式化有助于避免阻塞行为，并使定位收益或失效能够更容易地追溯到具体接口。

如果从数据流视角理解系统，过程可总结为以下七步：

1. 传感器首先获取原始激光点云与 IMU 数据，并同时输入 FAST-LIO 和 PointPillars。
2. PointPillars 产生当前帧的 3D 检测，指出附近存在哪些目标。
3. FAST-LIO 输出当前自位位姿，并持续提供高频运动预测。
4. 位姿桥接节点将这些定位输出转换为跟踪模块直接可用的格式。
5. MCTrack 结合“观测到了什么”与“自车如何运动”来完成目标关联、轨迹更新及速度估计。
6. 跟踪输出反馈给 FAST-LIO，以便对可能属于动态目标的点进行降权。
7. 同样的跟踪输出也被送入联合后端，以进一步精修自车轨迹。

这一七步描述已经说明了为什么系统必须组织为一个有向但延迟的闭环。轨迹在检测关联之前无法存在，动态点权重在跟踪消息存在之前也无法计算。因此，时刻 t 产生的跟踪状态影响的是后续扫描而非当前扫描。这一帧的延迟并非缺陷，而是保证闭环能够在线运行且不产生循环阻塞的关键。

3.1 FAST-LIO 前端

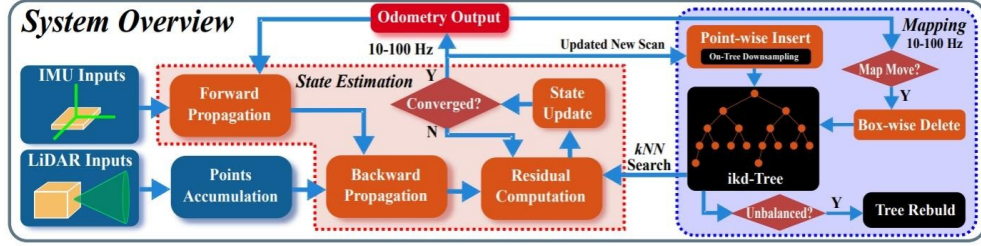


图 2: FAST-LIO2 前端架构，转载自 Xu *et al.* [1]。本项目中，来自 MCTrack 的对象信息被注入点云预处理管线，实现对动态点的自适应降权。

FAST-LIO 是整个系统的定位主干，也是上述数据流中的起点。它直接消费来自 KITTI rosbag 的激光点云与 IMU 数据，内部使用紧耦合的激光惯性估计管线输出稳定的自车位姿。本项目中，FAST-LIO 不仅产生全局一致的 `/Odometry`，还发布 `/Odometry_imu_predicted` 作为高频预测位姿，为后续跟踪与后端优化提供统一参考。

从模块职责看，FAST-LIO 在系统中扮演三个角色。首先，它为 MCTrack 提供可靠的自车位姿，使得 3D 检测在跨帧关联前能完成自运动补偿。其次，它为联合后端提供原始里程计约束，使后端优化不至于偏离前端估计过远。第三，通过 `preprocess.cpp` 及 `laserMapping.cpp` 中的跟踪对象回调路径，它接收 `/mctrack/tracked_objects`，将其转换为内部 `DynamicObject` 结构，并将目标运动属性映射为点级权重。因此，FAST-LIO 既是自运动信息的源头，也是对象反馈最直接地改变扫描级几何处理的地方。

从算法角度看，FAST-LIO 前端可以理解为一个结合了 IMU 传播与激光几何修正的迭代滤波器。设系统状态为 $\mathbf{x}_k$ ，IMU 输入为 $\mathbf{u}_k$ 。预测步可写为：

$$\mathbf{x}_{k+1|k} = f(\mathbf{x}_{k|k}, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{P}_{k+1|k} = \mathbf{F}_k \mathbf{P}_{k|k} \mathbf{F}_k^T + \mathbf{Q}_k, \quad (1)$$

其中 $\mathbf{P}$ 为状态协方差， $\mathbf{F}_k$ 为线性化状态转移矩阵。对于当前帧中的第 j 个点，FAST-LIO 进一步利用局部地图中的参考平面构建点到面的几何残差：

$$r_j = \mathbf{n}_j^T (\mathbf{R}(\mathbf{x}) \mathbf{p}_j + \mathbf{t}(\mathbf{x}) - \mathbf{q}_j), \quad (2)$$

并通过迭代更新来最小化:

$$\delta \mathbf{x}^{\star} = \arg \min_{\delta \mathbf{x}} \left(\sum_j \|r_j\|_j^2 + \|\delta \mathbf{x}\|_{\mathbf{p}_{k+1|k}^{-1}}^2 \right). \quad (3)$$

这些公式描述了 FAST-LIO 在本项目中的核心作用：IMU 提供高频连续预测，激光点云残差将位姿拉回几何一致的解。这就是为什么 FAST-LIO 既能稳定输出 /Odometry，又能充当 MCTrack 的时空参考。

为了将 FAST-LIO 的输出接入跟踪模块，项目实现了 `fastlio_pose_bridge.py`，它利用配置的机体系到雷达系外参将 `/Odometry` 转换为 `/mctrack/lidar_pose` 并以 TF 形式发布。这一显式桥接使得定位、跟踪与可视化保持一致的坐标系约定。FAST-LIO 在预处理每一帧扫描前会将最新的跟踪对象冻结为扫描局部快照，因此每个点都是针对一致的对象集合进行加权的，而不是针对中途变动的消息缓冲区。

3.2 PointPillars 对象观测模块

项目将 PointPillars 检测器 [2] 以 ROS 节点形式接入，具体实现位于 `MMDET3D/lo-cal/ros/nodes/kitti_pointpillars_bag_node.py`，因此能够直接消费 rosbag 中的 KITTI 点云。它的作用不只是输出原始检测，更是为定位增强链路提供统一、可溯源且带时间戳的 3D 对象观测消息。项目在 `catkin_ws/src/ME5400/msg/Detection3D.msg` 与 `catkin_ws/src/ME5400/msg/Detection3DArray.msg` 中定义了消息格式，包含帧 ID、类别、置信度、2D 框、3D 尺寸、雷达系位置和偏航角。因此，PointPillars 与 MCTrack 之间的通信不依赖任何私有第三方消息格式，而是建立在项目自定义的显式接口之上。

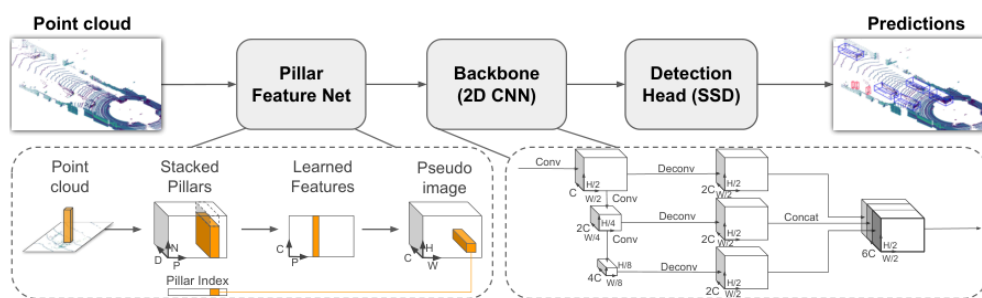


图 3: 来自原始 PointPillars 论文的网络架构。该网络由 Pillar Feature Net、2D CNN 主干和 SSD 检测头组成。图像转载自 Lang *et al.* 的 CVPR 2019 论文 [2]。

PointPillars 的本质是通过编码垂直柱 (pillars) 将稀疏 3D 点云转换为 BEV 伪图像, 然后应用 2D CNN 检测头。这把昂贵的 3D 卷积问题转化为了廉价的 2D 卷积, 这也是为什么 PointPillars 能够满足当前在线系统的实时性要求。

用公式表示，若第 l 个 pillar 包含点集 $\mathcal{P}_l = \{\mathbf{p}_i\}$ ，PointPillars 首先为每个点构造增强特征：

$$\tilde{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - \bar{x}_l, y_i - \bar{y}_l, z_i - \bar{z}_l, x_i - x_l^c, y_i - y_l^c], \quad (4)$$

其中 $(\bar{x}_l, \bar{y}_l, \bar{z}_l)$ 是 pillar 内点的均值， (x_l^c, y_l^c) 是 pillar 中心。经过 Pillar Feature Net 后，pillar 级特征可写为：

$$\mathbf{f}_l = \max_{i \in \mathcal{P}_l} \phi(\tilde{\mathbf{p}}_i), \quad (5)$$

随后将其散射到 BEV 伪图像中：

$$\mathbf{I}(u, v) = \begin{cases} \mathbf{f}_l, & (u, v) = \text{index}(l), \\ \mathbf{0}, & \text{否则}, \end{cases} \quad (6)$$

最后，检测头联合优化分类、框回归与朝向预测：

$$\mathcal{L}_{\text{det}} = \beta_{\text{cls}} \mathcal{L}_{\text{cls}} + \beta_{\text{loc}} \mathcal{L}_{\text{loc}} + \beta_{\text{dir}} \mathcal{L}_{\text{dir}}. \quad (7)$$

从工程视角看，这一设计服务于三个目的：保持标定一致性；维持较低检测门限以便后续通过跟踪与门控进行裁剪；暴露可被 MCTrack、可视化及评估重用的统一消息契约。PointPillars 有意不在内部解决跟踪问题，它的职责是发布具有时间戳和得分的几何 3D 假设，使后续系统能够把研究重点放在“这些对象信息如何服务定位”上。

3.3 MCTrack 对象状态估计模块与数据流细节

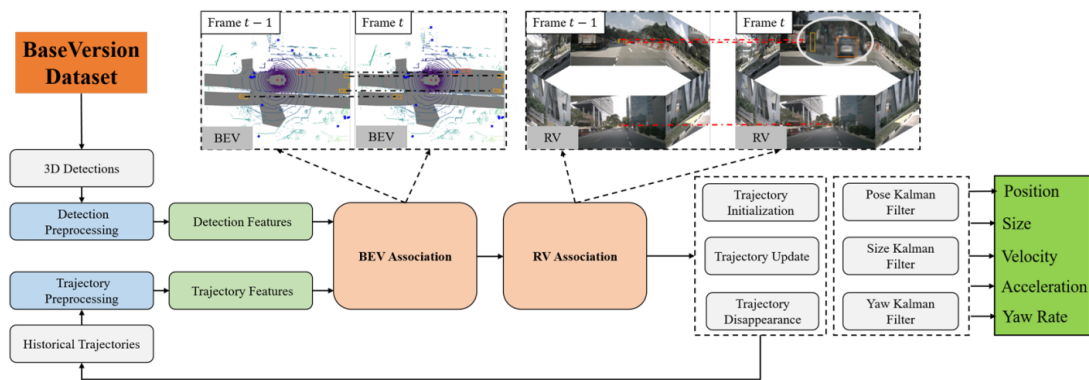


图 4: MCTrack 3D MOT 框架示意图，转载自 Wang *et al.* [3]。其输出不仅包含位置和尺寸，还包含速度、加速度、轨迹长度及相关运动属性。

在线 MCTrack [3] 节点实现在 `catkin_ws/src/ME5400/scripts/mctrack_online_node.py` 中。它默认订阅 `/mctrack/lidar_pose` 和 `/detection/lidar_detections`，内部维护位姿缓冲区和待处理检测队列，仅在合适时间戳的位姿可用后才处理检测。这种“检测

等待位姿、位姿驱动处理”的设计避免了纯按到达顺序处理导致的跨帧关联的时间错位。

MCTrack 同时发布可视化用的 `/mctrack/markers` 和下游优化用的 `/mctrack/tracked_objects`。在 `TrackedObject.msg` 中，除了对象位姿和尺寸，还记录了 `track_length`、`velocity`、`velocity_fusion`、`velocity_diff`、`velocity_curve` 和 `acceleration`。这些字段非常关键，因为随后的动态权重优化和联合后端门控均依赖运动属性而非仅仅框位置。

在当前的 `kitti_fastlio.yaml` 配置下，MCTrack 在线模式的主状态估计可近似为 2D 匀速卡尔曼滤波。若轨迹状态为：

$$\mathbf{x}_k = [p_x, p_y, v_x, v_y]^\top, \quad (8)$$

其预测模型为：

$$\mathbf{x}_{k|k-1} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_{k-1|k-1}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{z}_k = \mathbf{H}\mathbf{x}_{k|k-1} + \mathbf{v}_k, \quad (9)$$

随后执行标准卡尔曼更新：

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R})^{-1}, \quad \mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}\mathbf{x}_{k|k-1}). \quad (10)$$

在数据关联层，当前在线配置主要采用基于预测框的 3D 代价矩阵及匈牙利匹配：

$$\mathcal{A}^* = \arg \min_{\mathcal{A} \in \Pi} \sum_{(i,j) \in \mathcal{A}} C_{ij}, \quad C_{ij} = 1 - \text{RO-GDIOU}_{3D}(\hat{B}_i, B_j), \quad (11)$$

其中 $\hat{B}_i$ 为轨迹预测框， B_j 为当前检测框。换言之，FAST-LIO 首先通过自运动位姿稳定了跨帧几何，随后 MCTrack 通过运动预测与全局最优匹配将离散检测转换为连续轨迹。这是后续速度估计、动态权重与联合后端得以成立的基础。

缓冲与时间戳逻辑尤为重要。MCTrack 存储有限的 `pose_buffer` 和待处理检测队列。对于每个检测消息，它寻找检测时刻前后紧邻的位姿；若均存在则插值，否则短暂等待并回退到最近可用位姿。这是在线实时性与严格同步之间的实用折中。MCTrack 还保留了世界系 `pose` 与雷达系 `lidar_pose`，因为 FAST-LIO 需要雷达系版本进行点-框相交判断，而后端则需将世界系外推运动与自车状态进行对比。

总而言之，PointPillars 和 MCTrack 在系统中不仅是检测器和跟踪器。它们共同构成了为定位模块提供对象级几何与运动语义的动态观测链路。FAST-LIO 提供稳定的自运动参考，PointPillars 提供带时间戳的目标假设，MCTrack 将这些假设转化为可跨时间传播的轨迹状态。只有完成这一转换，动态权重或对象一致性残差才真正能够服务于

定位增强。

3.4 时间对齐、坐标系与失效模式

在优化的在线运行中，rosvbag 播放仅发布原始激光与 IMU 话题，而 PointPillars、FAST-LIO、MCTrack 及后端作为具有不同回调时序的独立 ROS 节点运行。因此，时间同步是一个运行时问题而非一次性标定问题。这也是为什么位姿缓冲、待检测队列以及带有一帧延迟的权重闭环是架构的结构性部分。

主要的运行时失效模式直截了当：**检测延迟**迫使跟踪器使用邻近而非真正同步的位姿；**漏检**缩短了轨迹并使运动估计不稳定；**轨迹碎片化**破坏了权重反馈和后端复用所需的连续性；**时间戳或坐标系不一致**会放大速度外推误差，甚至将接口 Bug 误认为算法失效。该架构降低了这些风险，但无法完全消除。

4 工程实现与系统集成

4.1 整体代码组织

仓库布局有意围绕集成的责任划分，而不仅仅是算法名称。对本报告最重要的顶级目录包括 catkin_ws/、MCTrack/、MMDet3D/、Scripts/ 以及 Results/。catkin_ws/ 包含系统的 ROS 原生部分：自定义消息、可启动脚本、配置文件以及修改后的 FAST-LIO 软件包。MCTrack/ 以相对自包含的形式保留了跟踪框架和配置，以便在线节点能导入并对其进行包装，而非重写跟踪逻辑。MMDet3D/ 包含将 PointPillars 包装进 ROS 流程所需的检测侧代码。Scripts/ 包含编排逻辑、辅助工具和一键入口。Results/ 则不是一个被动的数据堆放夹，它是报告所依赖的每一个图表和指标的标准实验归档库。

这种划分有两点好处。首先，它将沉重的第三方或依赖框架的代码与项目特定的集成代码分离开来，使得升级某个模块而不断掉其他模块变得更容易。其次，它紧贴用户的实际工作流。在实验过程中，用户很少直接编辑检测算子或卡尔曼更新；相反，用户会启动脚本、检查日志、检查 ROS 话题、记录轨迹以及对比结果文件夹。将仓库组织成与之匹配的形式可以减少阻碍，并缩短调试与评估过程中的反馈周期。当前课程项目的完整代码仓入口为 <https://github.com/XC-CN/ME5400>，报告中的脚本、结果目录与工程结构说明均以该仓库为基准。

表 2: 在线系统的代表性入口脚本及其功能

模块	代表性入口脚本	功能摘要
在线基线	<code>run_baseline.sh</code> <code>run_fastlio.sh</code>	执行纯 FAST-LIO 在线基线管线，并将基线轨迹记录在 <code>trajectory_baseline.txt</code> 中。
在线优化	<code>run_optimized.sh</code> <code>run_pointpillars_node.sh</code> <code>run_mctrack_online_node.sh</code> <code>run_joint_backend_ego.sh</code>	编排 PointPillars、MCTrack、开启权重优化的 FAST-LIO 以及联合后端，并输出优化前端轨迹 <code>trajectory.txt</code> 及后端轨迹 <code>trajectory_joint.txt</code> 。
一键在线对比	<code>run_all.sh</code>	顺序执行在线基线与在线优化管线，并将完整的运行时日志归档至结果目录。
评估与归档	<code>evaluate_trajectories.py</code> <code>odom_to_tum_recorder.py</code>	统一生成定量指标与可视化文件，包括 <code>metrics.json</code> 、轨迹误差对比图及指标汇总图。

表 3: 主要仓库目录及其工程角色

目录	代表性内容	工程角色
<code>catkin_ws/</code>	<code>src/ME5400/msg/</code> <code>src/ME5400/scripts/</code> <code>src/fast_lio/</code>	ROS 原生工作空间，用于存放自定义接口、桥接节点、MCTrack 在线包装、联合后端、轨迹记录及修改后的 FAST-LIO 前端。
<code>MCTrack/</code>	<code>config/kitti_fastlio.yaml</code>	封装了在线 ROS 节点所使用的跟踪框架以及运动/关联配置。
<code>MMDet3D/</code>	PointPillars ROS 包装及检测工具	存放使 PointPillars 能在项目数据和话题约定下运行的检测侧代码。
<code>Scripts/</code>	<code>run_all.sh</code> <code>run_baseline.sh</code> <code>run_optimized.sh</code> <code>evaluation/</code>	提供一键执行、编译/设置辅助、评估入口以及脚本化实验流程。
<code>Results/</code>	<code>metrics.json</code> , 轨迹, 图像, 日志	作为后续分析、报告撰写和回归检查中使用的序列级输出的标准归档。

4.2 在线系统设计

在在线模式下，`run_all.sh` 会顺序执行 `run_baseline.sh` 和 `run_optimized.sh`。这不仅是一个便捷包装脚本，更编码了实验的公平性原则。基线运行只启动 `roscore`、FAST-LIO 和 `rosbag` 播放，因此纯定位轨迹会被记录为 `trajectory_baseline.txt`。随后，优化运行再启动 PointPillars、MCTrack、处于 `both` 模式的 FAST-LIO 以及联合后端，同时由 `odom_to_tum_recorder.py` 将 `/joint_backend/odom` 记录为 `trajectory_joint.txt`。由于这些脚本面向同一序列 ID、使用同一数据源，并将输出归档到同一个结果目录中，因此它们构成的是受控对比，而不是松散拼接的独立实验。

在线优化脚本的控制流也经过专门设计，以尽量减少竞争条件。PointPillars 最先启动，因为它的模型初始化通常最慢。MCTrack 随后启动，并等待位姿和检测都已可用。接着启动带动态权重的 FAST-LIO，以便跟踪器在里程计出现后立刻开始填充自己的位姿缓存。最后，联合后端启动，轨迹记录同时开始。如果把这个顺序反过来，就会

出现一系列可以预见的问题：记录器可能在后端话题尚不存在时就开始工作；跟踪器可能在没有有效位姿的情况下积压检测；检测器也可能发布过晚，从而引发一批陈旧检测。因此，图 5 记录的不只是一个概念上的闭环，而是保持优化链条稳定运行的真实执行逻辑。

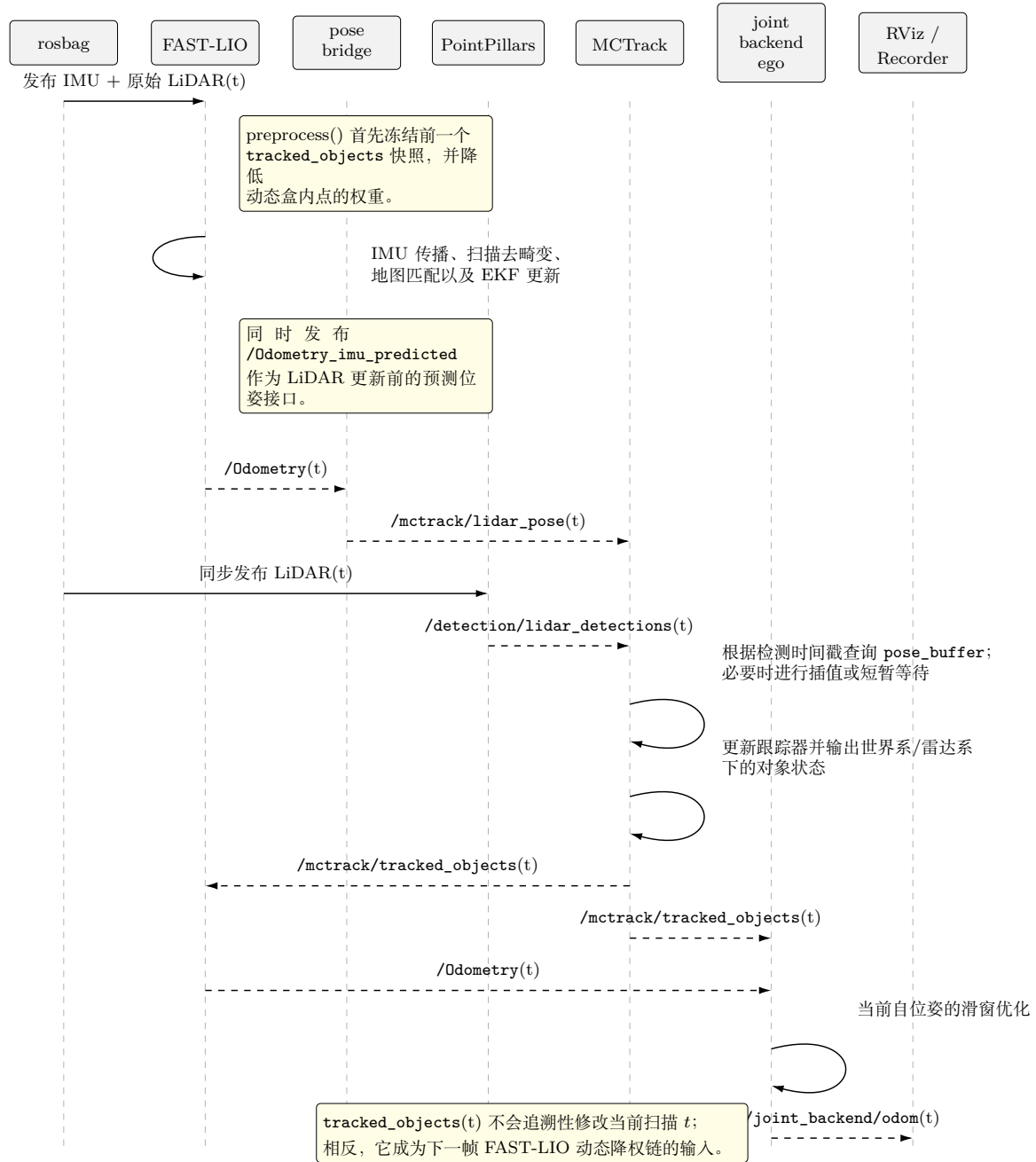


图 5: 在线优化链的执行级时序逻辑。当 FAST-LIO 处理当前 LiDAR 帧时，它使用的是前一帧冻结下来的 tracked_objects 快照。MCTrack 在当前帧产生的跟踪结果，主要会在下一帧进入动态降权链路。

如图 5 所示，在线链路并不是“当前帧检测立即修改当前帧点云”的同步闭环，而是一个带有一帧延迟的异步闭环。在 preprocess() 中，FAST-LIO 会先冻结上一时刻的 /mctrack/tracked_objects 快照，并对当前扫描中落入动态框的点进行软降

权。之后，当前时刻的 `/Odometry(t)` 会经由桥接节点传入 `MCTrack`，与 `PointPillars` 发布的 `/detection/lidar_detections(t)` 对齐并用于更新目标状态。新生成的 `/mctrack/tracked_objects(t)` 随后会同时发往 `FAST-LIO` 和联合后端，但它对 `FAST-LIO` 的影响要到下一帧才会体现。这种设计避免了帧内阻塞，减少了预处理阶段的锁竞争，也解释了为什么当前系统能够在保持在线可运行性的同时形成前端反馈闭环。

当 `rosbag` 播放结束后，脚本会将 `trajectory.txt`、`trajectory_joint.txt`、日志以及评估图统一归档到 `Results/<seq>_results/Online/`。这项设计的价值不只是“整理得更干净”，而是让后续的报告撰写或调试工作能够准确重建某个序列究竟发生了什么：运行了哪些脚本，对比了哪些轨迹，指标是多少，前端和后端的图像表现如何。当然，系统目前在话题层面仍然是松耦合的，各模块共享的是消息而不是统一图状态。但从工程角度看，显式消息传递正是当前原型可检查、且能在当前项目工作流内重跑的前提。

4.3 结果归档与脚本化实验管线

`Results/<seq>_results/Online/` 下的结果归档，最好被理解为一份正式的实验契约。每个在线序列目录都包含基线轨迹、优化前端轨迹、联合后端轨迹（若有）、纯文本报告、JSON 报告、对比图以及完整运行日志。这意味着报告中的定量分析并不依赖人工抄录的数字，也不依赖瞬时终端输出，而是依赖可以在同一仓库工作流内通过重新运行脚本再次生成的版本化产物。这里所说的“可重复运行”，指的是在当前项目设置下重新跑通同一套脚本化管线，并恢复同类别的指标、图表和日志，而不是声称环境无关、机器无关的字节级完全复现。在实践中，这正是后续章节能够展开讨论的基础：ATE、RPE、Tr、Rot、失效序列和各种图表，都来自同一个已归档的结果源。

评估链路进一步强化了这种工作流内可重复运行性。`evaluate_trajectories.py` 会读取记录好的 TUM 轨迹，将其与 KITTI 真值进行关联，计算 ATE 和 RPE，对轨迹进行可视化对齐，并在匹配路径长度足够时计算 KITTI 风格的相对平移和相对旋转指标。因为每个基线与优化的对比都使用同一套评估脚本，报告避免了课程项目中常见的问题，即不同序列被用略有差异的方法分别评估。即便不同序列可用的有效轨迹跨度不同，评估规则本身仍然保持一致。

这种工作流层面的可重复运行性并不是附带便利，而是项目智力贡献的一部分。一个声称带来了定位增益，却不能重新运行自己的序列、恢复自己的指标、解释自己的失败的课程项目，其价值非常有限。相比之下，当前系统能够通过固定脚本启动，以标准格式记录轨迹，重建图表，并将报告中的每一条结论追溯到明确的归档产物。这也是为什么仓库工作流内的工程可重复运行性应该被视为一项一等成果，而不是附录级别的细节。

4.4 ROS 话题、消息接口与集成困难

本仓库中的模块边界主要通过 ROS 话题和项目自定义消息来维持。原始输入为 `/kitti/velo/pointcloud` 和 `/kitti/oxts/imu`。FAST-LIO 发布 `/Odometry` 和 `/Odometry_imu_predicted`。桥接节点转发 `/mctrack/lidar_pose`。PointPillars 发布 `/detection/lidar_detections`。MCTrack 发布 `/mctrack/tracked_objects` 和 `/mctrack/markers`。后端发布 `/joint_backend/odom`。每个话题都对应清晰的语义职责，而自定义消息的存在保证了检测器输出、目标跟踪状态和后端输入在整条链路中都保持可解释。

自定义消息对避免隐性耦合尤其重要。`Detection3D.msg` 存储帧索引、类别、置信度、2D 框、3D 尺寸、雷达系位置和航向角。`TrackedObject.msg` 在此基础上扩展出更适合反馈使用的目标状态，包括轨迹长度、世界系位姿、雷达系位姿、融合速度、速度差、曲率速度以及加速度。正是这种显式结构，使动态权重模块和联合后端都成为可能。动态权重代码可以直接读取雷达系位姿、轨迹长度、得分与加速度；后端则可以将局部观测与世界系预测直接进行比较，而无需从一个不透明的跟踪器内部对象中反向推断状态。换言之，消息层正是本项目把算法内部细节转化为稳定跨模块契约的地方。

即便如此，系统集成仍然困难重重。依赖隔离是一项挑战：MMDet3D、ROS、MC-Track 和 FAST-LIO 都有各自的运行时假设、环境变量和构建方式。启动顺序是另一项挑战，因为整个优化链条对 rosbag 播放前检测器、跟踪器和后端是否已准备就绪非常敏感。日志记录是第三项挑战，因此当前脚本会把完整输出重定向到各序列日志中。结果命名与归档则是第四项挑战：如果轨迹文件没有被稳定地重命名并移动到正确位置，后续评估就可能在无声无息中对比错误的结果。单看这些问题都不算“高深”，但它们共同决定了一个集成机器人系统到底像一个受控实验，还是只是一个不稳定的演示。

5 动态权重优化

5.1 设计动机与代码位置

这一模块的核心设计问题其实很直接：当一个被跟踪到的目标覆盖了点云的一部分区域时，系统应该保留这些点、彻底删掉这些点，还是连续地削弱它们的影响？本项目当前选择的是第三种方案。它不把动态点处理看成一个二元语义分割问题，而是把被跟踪目标视为一种证据，表明附近几何结构的可靠性应该根据其运动状态和置信度被连续地下调。从几何和工程两个角度看，这种做法都是合理的。几何上，硬删除可能会在交通场景中移除过多有用结构，尤其是当大型车辆占据了激光扫描很大一部分，而路边静态结构本来就不丰富时。工程上，硬删除对误检、不稳定框和部分遮挡都很脆弱；一个稍微偏掉的框就可能删除大量本来有效的静态点。软降权更保守，因此更适合作为第一版集成原型。

当前实现主要位于 `catkin_ws/src/fast_lio/src/preprocess.cpp`，而跟踪目标

到内部表示的转换逻辑位于 `catkin_ws/src/fast_lio/src/laserMapping.cpp`。当 FAST-LIO 收到 `/mctrack/tracked_objects` 后，每个对象都会被转换成一个内部 `DynamicObject`，其中保存该对象的雷达系中心、包围盒半尺寸、局部偏航表示、时间戳以及一个标量权重。这里有一个关键工程选择：转换使用的是对象在雷达坐标系下的位姿，而不是世界坐标系位姿，因为点是否落入框内的判断必须在当前 LiDAR 坐标系下进行。也因此，这个降权模块并不是依赖某种抽象全局地图，而是紧贴真实预处理流程工作的。

与更早的版本相比，当前代码中的动态降权默认参数已经被明确固定为：

- 参考速度 $v_{\text{ref}} = 10.0$;
- 参考加速度 $a_{\text{ref}} = 4.0$;
- 速度惩罚系数 $\alpha_v = 0.5$;
- 加速度惩罚系数 $\alpha_a = 0.3$;
- 检测得分惩罚系数 $\alpha_s = 0.2$;
- 最小权重 $w_{\text{min}} = 0.2$;
- 包围盒边界扩张 $xy = 0.5 \text{ m}, z = 0.5 \text{ m}$;
- 对象超时阈值 1.0 s ，以及最小轨迹长度阈值 3 。

每一个参数都对应一个工程折中。参考速度和参考加速度定义了“多快/多激烈才算明显动态”；最小权重阻止系统退化成硬删除；包围盒边界扩张用于补偿检测框边界不够精确的问题；超时与最小轨迹长度阈值则用来防止陈旧轨迹或尚未成熟的轨迹影响当前扫描。

5.2 权重计算

假设一个被跟踪目标具有速度 v 、加速度 a 和检测得分 s 。代码首先对这些量进行归一化：

$$\hat{v} = \text{clamp}\left(\frac{v}{v_{\text{ref}}}\right), \quad \hat{a} = \text{clamp}\left(\frac{a}{a_{\text{ref}}}\right), \quad \hat{s} = \text{clamp}(s). \quad (12)$$

然后构造惩罚项

$$p = \alpha_v \hat{v} + \alpha_a \hat{a} + \alpha_s \hat{s}, \quad (13)$$

并据此计算动态权重

$$w = \text{clip}(1 - p, w_{\text{min}}, 1). \quad (14)$$

这个解释是直观的。目标越快、加速度越大、检测得分越高，其对应点的权重就越低。如果一个轨迹很短、几乎静止，或者置信度本身就弱，系统就会保留更大的权重。

轨迹长度门控尤其重要：如果 `track_length` 低于阈值，`weightFromAttributes()` 会直接返回 1.0，也就是说这个目标还没有资格影响点云。

之所以选择速度、加速度和得分作为输入，是因为这些正是跟踪器当前已经能够较可靠输出的信息。这还不是一个完整的概率不确定性模型，但它是一个从 `TrackedObject.msg` 中已有的可解释运动属性出发构造出来的、足够实用的启发式方法。

5.3 从跟踪对象到点级权重

从跟踪对象到点级权重的转换，发生在一次扫描局部的处理管线中。跟踪对象回调首先读取 `lidar_pose.position`、加了边界冗余的半尺寸、紧凑的偏航表示，以及由融合速度、加速度、检测得分和轨迹长度共同计算出来的标量权重，并将其转成内部表示。当新扫描进入 `process()` 时，FAST-LIO 会把最新的跟踪对象冻结为一个活动快照。此后，每个点都会调用 `computePointWeight()`，检查自己是否落入任何未过期的动态框内，并取所有覆盖它的动态框中的最小权重。也就是说，`tracked_objects(t)` 并不会回溯性地修改时间 t 的那一帧扫描；它会作为下一次预处理所使用的一致对象集合。

这个权重会直接在点云预处理中生效。对于每个输入点，如果 `computePointWeight()` 发现其与动态框重叠，那么该点的 `intensity` 会被更新为原始强度与动态权重中的较小者：

$$I'_j = \min(I_j, w_j), \quad (15)$$

其中 I_j 是原始点强度， w_j 是计算得到的动态权重。这种实现复用了 FAST-LIO 已有的点表示，把“几何可靠性”信息无缝带入后续流程，而不需要重写配准主干。

与直接删除动态点相比，这一策略对框边界误差、检测抖动和“部分动态区域”都更稳健；当静态背景本来就少时，它仍能保留一部分几何支撑；同时，它还能区分“一个明显在动且轨迹已成熟的目标”和“一个刚刚出现、尚未确认的目标”，而不是做一个生硬的语义开关。这种分级行为，也是为什么当前前端闭环明显比后端更稳定的原因之一。

5.4 边界情况与参数敏感性

若考察若干边界情况，就能更清楚地理解为什么当前系统更适合软降权而不是硬删除。**误检与遮挡引发的框不稳定**都可能覆盖静态结构，或者在大小上发生抖动；在硬删除策略下，这些误差会把几何直接抹掉，而软降权只会在轨迹已成熟、运动判定成立之后削弱其影响。**时间上不确定的情况**，例如暂时停下的车辆或已经陈旧的轨迹，也会通过得分、加速度、最小轨迹长度和超时机制被更保守地处理。

虽然本项目没有做形式化的参数扫描，但现有归档序列已经足以支持一个定性的敏感性讨论。如果 v_{ref} 设得太低，或者 w_{min} 设得太小，这个模块的行为就会过于接近

硬过滤；如果 v_{ref} 太高，或者 w_{min} 太大，动态降权又会弱到几乎不起作用。当前这组参数看起来是一个实用的折中：它足够强，能够在 0010 和 0020 这类易漂移序列上带来明显改善；同时又足够保守，不会在全部七个在线序列中系统性地引入退化。

从整体效果上看，这个模块改善的是扫描匹配与局部地图一致性，而且并不要求目标信息已经精确到足以支撑强后端约束。这种平衡，正是动态降权成为当前系统最稳定增益来源的原因。

6 联合后端优化

6.1 仅自车滑窗问题的提出

联合后端的实现位于 `catkin_ws/src/ME5400/scripts/joint_backend_ego_node.py`，配置文件位于 `catkin_ws/src/ME5400/config/joint_backend_ego.yaml`。它的定位本身就是过渡性的。当前目标并不是一步到位地解决完整的对象中心化 SLAMMOT 问题，而是用可控的工程成本先验证一个更窄的问题：即便不把对象状态纳入联合优化，稳定的跟踪目标是否已经足以为自车轨迹提供额外结构，从而带来修正价值？这个问题之所以重要，是因为它决定了项目是否值得继续向更复杂的后端投入更多工程。也正因此，当前的仅自车后端并不是最终形态，而是一个诊断性的过渡台阶。

当前配置把这种折中写得很明确。默认窗口长度为 8 帧。对象因子系数为 $\lambda_{\text{obj}} = 0.12$ 。得分阈值为 0.6，轨迹长度阈值为 5，允许的最大时间戳差为 0.1s，允许的最大速度跳变为 2.0 m/s。对象最小权重为 0.03。后端使用尺度为 0.5 的 Huber 鲁棒损失，以 10 Hz 优化；若 FAST-LIO 输入协方差无效，则回退到 [0.20, 0.20, 0.10]。这些选择都不是随意拍脑袋定下来的。它们共同定义了一个对对象信息“何时可信”非常保守、同时对状态规模保持轻量的后端原型。

窗口中每一帧的优化变量写作

$$\mathbf{p}_i = [x_i, y_i, \psi_i]^\top, \quad (16)$$

也就是说，每一帧只保留平面位置和偏航角。这一选择固然受限，但从工程角度看是合理的。第一，当前后端所利用的跟踪对象信息在运动语义上本来就主要是二维的：融合速度、关联稳定性等信息主要体现在地平面内。第二，只保留最关键的自车变量，能够让优化规模足够小，从而满足在线反复求解的需求，不至于成为系统的主要算力瓶颈。显然，这样做的代价是对象误差无法被显式估计；如果某个轨迹本身就是错的，后端内部没有任何对象状态变量可以吸收该误差。

因此，先采用这种仅自车形式，本质上是一个有风险控制策略。若一开始就做完整联合图，必须同时处理对象位姿、对象速度、数据关联不确定性、边缘化策略，以及更谨慎的运动模型设计。相比之下，当前这个后端可以直接利用 MCTrack 已经提供的输出进行实现、测试和压力分析。所以，把它称作“后端方向的原型”比称作“已

经完成的后端方案”更准确。

6.2 残差建模与门控策略

联合后端包含两类主要残差。第一类是里程计一致性残差：相邻 FAST-LIO 帧之间的相对位姿被视为滑窗中的主约束，以确保优化结果不会偏离原始里程计太远。第二类是对象观测一致性残差：对于每一帧，后端会选取时间上最近的跟踪快照，并且只保留那些通过得分、轨迹长度、速度跳变和时间同步门控的对象。对于每个保留下来的对象，代码会先依据其速度将世界系位置外推到当前帧时间戳，再把该位置投影到优化所使用的自车局部坐标系中，得到预测观测 z_{ik}^{pred} 。测量观测 z_{ik}^{meas} 则来自 `TrackedObject` 中记录的 LiDAR 局部位姿。因此，对象残差写作

$$r_{ik}^{\text{obj}} = z_{ik}^{\text{pred}} - z_{ik}^{\text{meas}}. \quad (17)$$

整体优化目标为

$$\min_{\{\mathbf{p}_i\}} \sum_{i=2}^N \|r_i^{\text{odom}}\|_{W_i}^2 + \sum_{i=1}^N \sum_{k \in \mathcal{V}_i} w_{ik} \rho\left(\|r_{ik}^{\text{obj}}\|_2^2\right), \quad (18)$$

其中 $\rho(\cdot)$ 是 Huber 或 Cauchy 鲁棒核， $\mathcal{V}_i$ 表示通过门控的对象集合。在当前代码里，对象因子权重大致写作

$$w_{ik} = \max\left(w_{\min}^{\text{obj}}, \sqrt{\lambda_{\text{obj}}} \cdot (0.5 \hat{s}_{ik} + 0.5 \hat{\ell}_{ik})\right), \quad (19)$$

也就是说，它由归一化检测得分和归一化轨迹长度共同决定。

这种残差结构是刻意不对称的。里程计残差被当作滑窗的稳定骨架。对象残差则被视为可选修正项，其作用是在对象证据成熟且一致时，对解施加轻微推动。因此，后端并不要求被跟踪对象主导整个估计；它只是希望对象在条件合适时，为“自车应当如何相对于环境定位”提供第二种一致性证据。对一个原型系统来说，这种不对称性是合理的，因为它限制了错误轨迹可能造成的破坏幅度。至少在理论上，即便对象因子不完美，里程计链条也应当依然能够锚定滑窗。

在实践中，这个理论能否成立，取决于门控策略本身。当前后端会首先丢弃那些得分低于阈值、轨迹长度过短、速度跳变过大，或者与当前里程计状态时间差过大的对象。这些条件几乎直接对应了“复用对象信息”最常见的失败通道。低得分意味着检测质量不确定；短轨迹意味着时间证据尚不成熟；大速度跳变往往意味着数值更新不稳定或语义关联本身就不可靠；时间戳错位过大则会让速度外推失去物理意义。因此，这套门控并不是任意保守，而是把“哪些对象已经成熟到足以作为弱移动锚点”这一工程判断明确编码了出来。

6.3 滑窗更新逻辑与实际行为

当前后端由定时器回调驱动，而不是对每一条消息都立即优化。它会把近期里程计状态与跟踪快照保存在环形缓冲区中，当帧数足够时构造最新滑窗，并避免对同一个窗口末端时间戳重复求解。优化结果会被用来修正最新原始里程计位姿，同时保留 FAST-LIO 的滚转角和俯仰角，只调整平面位置与偏航角。因此，这个后端在实际行为上更像一个在线修正器，而不是完整的平滑引擎。

这也正好解释了后文实验中出现的“后端表现复杂”的现象。当对象信息足够干净，而基线漂移又足够明显时，后端可以提供有价值的低频修正；而当对象信息存在噪声时，由于窗口较小且对象状态并未作为变量一起优化，少数错误残差仍可能直接显现到最终结果中。

6.4 联合后端的价值与局限

这个后端的工程价值在于，它验证了一个关键命题：即便对象状态还没有作为显式优化变量引入，稳定的跟踪对象依然可以作为临时的“移动锚点”，为自车轨迹施加额外约束。换言之，系统已经不再把动态目标仅仅视作需要滤掉的噪声，而是开始把它们视作能够进入估计问题的正信息来源。从这个意义上说，当前后端是一个“方向验证器”。它说明了，被跟踪对象不仅能用于前端降权，也能在后端中作为正向约束。

但它的局限同样清晰。由于对象状态来自跟踪器的点估计，并且通过速度外推被继续传播，目标速度、检测框或关联结果中的任何噪声，都有机会反向注入到自车位姿中。当前后端也缺乏更丰富的因子类型：没有显式对象状态变量，没有超越“得分 + 轨迹长度”的更强置信度模型，也没有对数据关联不确定性的复杂表示。再加上跟踪器当前隐含的运动模型仍然较简单，因此外推出的对象位置并不总是足够可信，未必真的适合拿来约束自车运动。

Sequence 0009 是最清晰的失效案例。在在线模式下，基线 ATE RMSE 为 1.6831 m，前端闭环略微改善到 1.6675 m，但联合后端却把 ATE RMSE 恶化到 3.3082 m，同时 RPE RMSE 从 0.1787 飙升到 1.9514。最合理的解释并不是“对象信息原则上没用”，而是该序列中若干薄弱环节同时朝不利方向叠加了。速度外推可能在短时间目标轨迹上迅速积累误差；时间戳即便勉强通过门控，也可能降低外推状态的物理意义；基于得分、轨迹长度和速度跳变的门控也不足以屏蔽所有有害因子。一旦这些残差进入一个小型、仅自车的滑窗，而优化器又没有对象状态变量和更强的不确定性模型去吸收冲突，失败就是自然结果。因此，0009 更像一个强诊断例，而不仅仅是“后端偶尔不稳定”的笼统证明。

因此，对这个后端的结论应当是克制而明确的。方向本身是对的，因为某些序列确实表现出了有用的漂移修正，而且当前公式已经证明对象信息可以进入自车优化问题。但当前这个仅自车实现仍然只是一个工程原型，而不是鲁棒的联合后端。合理的演化路径其实已经很清楚：更强的置信度建模、显式引入对象状态变量、扩展因子类型，而最

终则是走向一个真正对象中心化的后端，在其中自车运动和对象运动被共同估计，而不是一个只是被另一个拿来修正。

7 实验分析

7.1 实验设置与结果来源

本报告中的实验统一建立在已经归档于 `Results/` 目录下的在线结果之上。分析的在线序列包括 0003、0006、0007、0009、0010、0013 和 0020。所有数值均直接取自各自的 `metrics.json` 文件，而文中讨论的图像则对应 `evaluation_result.png`、`evaluation_joint_backend.png` 和 `ablation_bar.png`（指标汇总柱状图）。因此，本节并不引入在写报告期间新跑出来的额外实验，而是对已经通过归档的脚本化实验管线生成的结果进行归纳和解释。

在线模式下共比较三种配置：Baseline、FAST-LIO+MCTrack 和 Joint Backend。Baseline 指纯 FAST-LIO。FAST-LIO+MCTrack 在此基础上同时引入位姿辅助跟踪与跟踪驱动的动态降权，并直接评估前端优化后的轨迹。Joint Backend 则进一步评估记录自 `/joint_backend/odom` 的仅自车后端输出。这样的排列顺序在分析上非常重要，因为它把“前端闭环带来的增益”和“后端额外带来的增益或损失”清晰分离开了。但也需要明确指出：当前归档实验验证的是这一前端闭环组合的整体有效性，而不是对“位姿辅助跟踪”和“动态降权”两项机制分别做了严格消融；因此，后文涉及前端收益来源的表述，都应理解为针对组合机制的判断，而非对某一单独环节的完全因果归因。

本报告使用的评价指标各有侧重点，不应被混为一谈。**ATE RMSE** 衡量与真值对齐后整条轨迹的全局偏差均方根，因此最能反映累计漂移和整体路径一致性。**RPE RMSE** 衡量短时间尺度上的相对运动偏差，因此对局部帧间稳定性更敏感。**KITTI 相对平移误差 Tr** 统计固定路径长度上的平移相对误差百分比，而 **KITTI 相对旋转误差 Rot** 则统计每 100 m 的角度漂移。一个方法完全可能改善 ATE 却不改善 RPE，或者改善平移误差却不改善旋转误差。这一点对理解当前后端尤其重要，因为它有时更像一个低频漂移修正器，而不是一个在所有局部尺度上都更优的优化器。

这七个序列覆盖了一组具有代表性的运行条件。0010 和 0020 代表了高动态或高漂移场景，在这类场景中前端反馈有充分空间展示价值。0003 和 0006 是更短或更中等强度的场景，其中前端能改善，但后端可能产生过修正。0007 和 0013 代表较为稳定的场景，改善幅度有限但基本一致。0009 则是关键失效案例，后端在其中显著恶化。按照场景属性来组织分析，比把每个序列孤立地当作一个数字更有信息量。

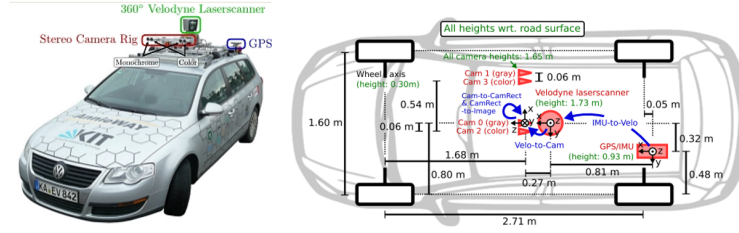


图 6: KITTI 数据采集平台，转载自 Geiger *et al.* [4] 相关的基准材料。本报告中的所有实验都基于已归档的 KITTI Tracking 结果。

7.2 在线实验：汇总结果

表 4 汇总了全部在线序列的 ATE 和 RPE。最重要的整体观察非常直接：FAST-LIO+MCTrack 在全部 7 个所选在线序列上都改善了 ATE RMSE。这种 7/7 的表现，比任何单个“大成功案例”都更有说服力，因为它表明前端闭环在这一组选定的评估切片上持续带来正收益。相比之下，Joint Backend 只在 4 个序列上进一步降低了 ATE，其中还有两个改善几乎可以视为持平。因此，即便只看这 7 个序列，后端也没有表现出与前端同等级别的跨序列稳定性。

表 4: 在线 ATE/RPE 结果汇总。改善百分比均相对 Baseline 计算；正值表示误差降低。

序列	Baseline ATE	F+M ATE	F+M 改善	Joint ATE	Joint 改善	RPE RMSE (B/F/J)
0003	0.5294	0.4513	+14.75%	0.5875	-10.98%	0.2049 / 0.1809 / 0.2393
0006	0.5238	0.4710	+10.09%	0.5576	-6.45%	0.1324 / 0.0869 / 0.1677
0007	2.3128	2.2393	+3.18%	2.2393	+3.18%	0.1716 / 0.1683 / 0.1683
0009	1.6831	1.6675	+0.93%	3.3082	-96.56%	0.1787 / 0.1651 / 1.9514
0010	3.7651	0.8680	+76.95%	0.9632	+74.42%	0.2774 / 0.2281 / 0.2334
0013	0.6047	0.5816	+3.82%	0.5816	+3.82%	0.1495 / 0.1392 / 0.1392
0020	11.7323	6.7938	+42.09%	6.8829	+41.33%	0.2250 / 0.2245 / 0.2270

这张 ATE 表也说明了为什么不能只看“平均改善”来评价后端。0010 是前端和后端都明显成功的序列，但 0009 却是一次灾难性的后端失败。因此，系统层面的真正结论是：前端增益稳定，而后端增益是有条件的。RPE 列进一步强化了这一点：前端往往能改善或至少保持局部稳定性，而当对象信息不可靠时，后端则可能引入明显的局部不一致。

表 5 给出了在线模式下的 KITTI 相对平移误差和旋转误差。这里依然可以看到，前端闭环的行为比后端更稳定。FAST-LIO+MCTrack 在 7 个可评估序列中的 6 个上改善了相对平移误差，在其中 4 个上改善了相对旋转误差；而 Joint Backend 只在 3 个序列上改善了平移误差，并且经常会让相对旋转误差变差。这与我们对当前后端的预期完全一致：它有时能纠正长时间尺度上的漂移，但还不能保证局部几何更平滑。

表 5: 在线 KITTI 相对误差汇总。Rot 的单位为 deg/100m, 顺序为 Baseline / F+M / Joint。

序列	Baseline Tr(%)	F+M Tr(%)	F+M 改善	Joint Tr(%)	Joint 改善	Rot (B/F/J)
0003	1.4794	1.3516	+8.63%	1.6856	-13.94%	1.1509 / 1.1569 / 1.3699
0006	4.1614	4.0605	+2.42%	4.3990	-5.71%	12.8416 / 11.6291 / 16.0675
0007	3.9042	3.9267	-0.57%	3.9267	-0.57%	4.9702 / 5.0234 / 5.0234
0009	2.5630	2.5281	+1.36%	4.2200	-64.65%	3.2425 / 3.2294 / 3.7550
0010	1.3884	0.9773	+29.61%	1.1140	+19.76%	1.1274 / 1.1368 / 1.5713
0013	2.4374	2.3741	+2.60%	2.3741	+2.60%	2.3026 / 2.2419 / 2.2419
0020	3.2245	2.3967	+25.67%	2.4302	+24.63%	1.1894 / 1.1748 / 1.2326

在所有在线结果中, 前端最强的两个成功案例是 0010 和 0020。对于 0010, ATE RMSE 从 3.7651 m 大幅下降到 0.8680 m, 改善幅度达到 76.95%。对于 0020, ATE RMSE 则从 11.7323 m 降至 6.7938 m, 改善 42.09%。这不是边缘性修补, 而是显著改善。它说明, 当基线漂移很大、动态目标又明显扰动点云时, 可靠位姿支持下的跟踪加上软动态点降权, 确实能够大幅改善定位。图 7 和图 8 进一步展示了 0020 的这一现象: 前者显示优化后的轨迹明显比基线更贴近真值, 后者展示了同一序列上的动态车辆跟踪效果。

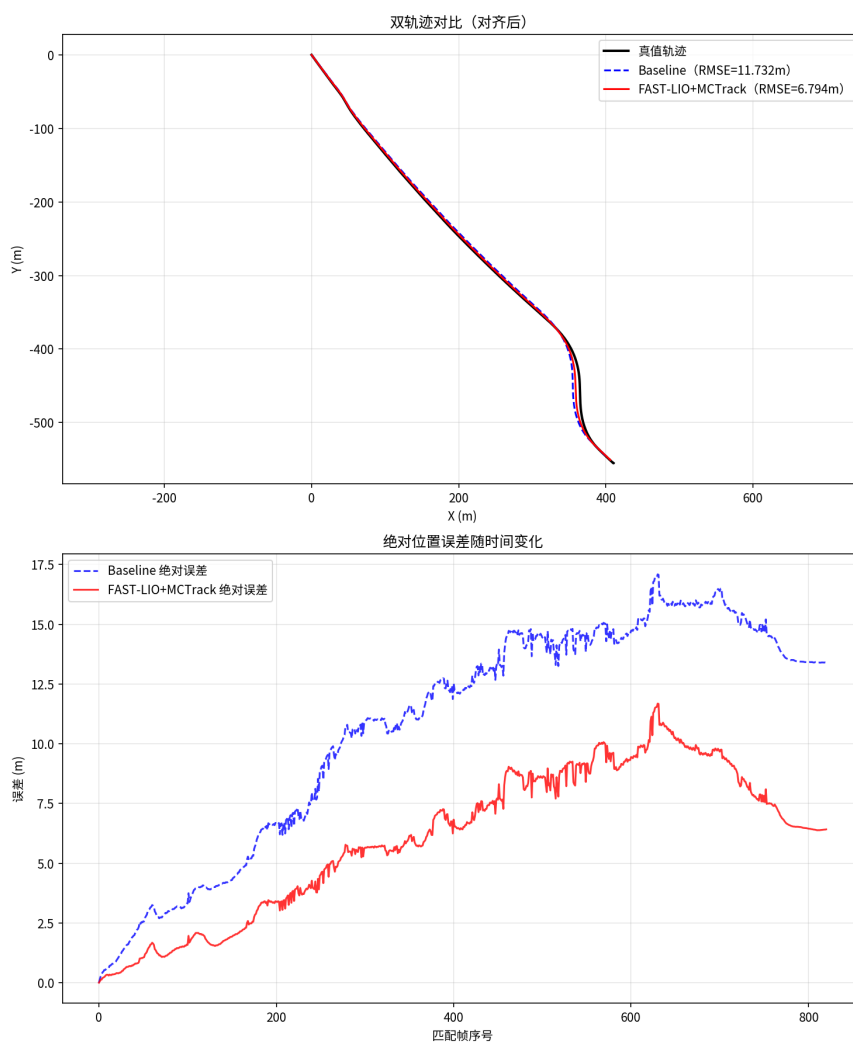


图 7: Sequence 0020 的在线轨迹评估结果。红色的 FAST-LIO+MCTrack 轨迹明显比蓝色基线更接近真值。

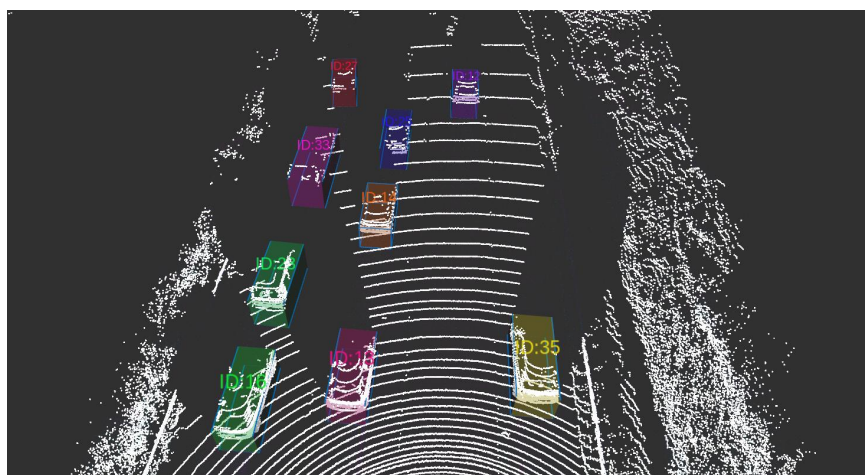


图 8: Sequence 0020 的动态车辆跟踪可视化结果。

7.3 综合分析与失效模式

高动态且基线漂移显著的场景：Sequences 0010 与 0020。这两个序列最能展示当前系统前端的价值。对于 0010，基线已经存在显著全局偏差，而前端闭环将 ATE RMSE 从 3.7651 m 降到 0.8680 m，同时还把 RPE RMSE 从 0.2774 降到 0.2281。联合后端相对于基线仍然是好的，但略差于前端输出，说明大部分收益已经在后端之前被前端捕捉到了。0020 展现出类似但时间尺度更长的模式。前端闭环把 ATE RMSE 从 11.7323 m 降到 6.7938 m，并将 KITTI 平移误差从 3.2245% 改善到 2.3967%。后端保住了大部分全局收益，但 RPE 几乎没有进一步改善。最合理的解释是：这些序列同时受到动态场景污染与全局漂移的影响，因此更好的前端降权直接带来了大收益，而后端已经没有太多干净的额外信息可供继续利用。

中等动态场景：Sequences 0003 与 0006。这两个序列体现了一个更微妙但同样重要的模式。前端闭环同时改善了 ATE 和 RPE：0003 的 ATE RMSE 改善了 14.75%，0006 改善了 10.09%，且 0006 的 RPE 从 0.1324 显著降到 0.0869。但在这两个序列中，后端都让性能退化，不仅把 ATE 推回到接近甚至高于基线，还明显恶化了 RPE 和 KITTI 相对误差。这说明前端改善并不是“因为序列太容易”；前端在中等场景下同样有效。真正的问题在于后端出现了过修正：当基线和前端已经足够好时，少量噪声对象因子就足以带来净负收益。换句话说，当前后端还缺少一个足够强的激活判据，用来判断“对象驱动修正究竟值不值得冒险启用”。

低动态或静态结构丰富的场景：Sequences 0007 与 0013。这两个序列的价值在于，它们测试了提出的方法在问题并不强动态时是否仍然安全。答案对于前端来说基本是肯定的。0007 的 ATE 从 2.3128 m 小幅改善到 2.2393 m，0013 也带来了温和但一致的 ATE 与 RPE 改善。后端在这两个序列中几乎与前端重合，这说明对象驱动修正既没有带来额外大收益，也没有造成严重破坏。这其实是一个有价值的结果：它说明前端降权闭环并不依赖极端动态场景才能稳定工作，同时也说明当静态结构本身已经足够约束轨迹时，当前后端没有太多正向杠杆可用。

后端失效案例：Sequence 0009。0009 值得单独拿出来讨论，因为它的失败在性质上与前面那些“中等退化”完全不同。前端闭环仍然带来了微弱但真实的改善：ATE RMSE 从 1.6831 m 降到 1.6675 m，RPE RMSE 从 0.1787 降到 0.1651。随后后端几乎崩溃：ATE RMSE 升到 3.3082 m，ATE 最大误差达到 40.3063 m，KITTI 平移误差从 2.5630% 恶化到 4.2200%，RPE RMSE 也跳升到 1.9514。这种模式说明局部但严重的不一致被直接注入了自车估计中。最可能的根因正如后端章节分析所述：速度外推误差、门控不够严格，以及里程计与跟踪对象之间的时间错位。0009 因而成为一个锚定性的反例，它证明当前后端不只是“没有前端好”，而是在某些条件下会主动破坏轨迹稳定性。

7.4 从现象到解释

将上述分组结果综合起来，可以得到一个较为清晰的机制解释。当基线主要受到动态目标污染前端匹配的影响时，当前系统会稳定获益，因为“位姿辅助跟踪 + 跟踪反馈降权”构成的前端闭环很可能在配准和建图误差积累之前打断了这一机制。这就是为什么前端闭环改善了全部 7 个所选序列，并且在高漂移动态场景中收益最大。需要强调的是，现有实验并未把前端闭环内部的两项机制彻底拆开评估，因此这里更稳妥的结论是：前端组合配置作为整体是有效的，而不是已经被严格证明收益主要且唯一来自某一个单独模块。相反，当基线已经相对稳定时，后端的剩余改善空间变小，而通过被跟踪对象施加的额外约束却更容易把噪声重新注入自车位姿。

ATE 和 RPE 趋势之间的差异也很有启发性。在某些序列上，后端能够保留更好的全局路径形状，却几乎不改善甚至恶化 RPE，这与“它更像低频修正器而非全面更优的局部优化器”这一解释是一致的。前端之所以稳定，是因为它对对象信息的使用方式是局部且保守的；后端之所以脆弱，则是因为它把跟踪对象信息转成了更强的自车约束，因此对同步质量、置信度建模和运动预测精度提出了更高要求。

7.5 实验公平性与报告纪律

最后，本报告的实验结论之所以具有说服力，还因为所有已报告序列都采用了统一的比较协议。基线、前端优化和后端优化使用的是同一归档数据源、同一轨迹格式、同一评估脚本和同一结果目录约定。尤其是在面对“小改善”和“大失败”同时出现的情况下，这一点非常关键，因为它显著降低了“趋势只是由后处理差异引起”的可能性。

因此，本报告的核心实验结论在当前实际测试范围内是明确的。在这 7 个所选在线序列和当前归档工作流下，项目已经验证：利用对象级动态信息进行前端动态降权，能够在 KITTI 多序列场景中稳定改善 SLAM 定位误差。这个对象感知前端定位增强链路正是当前系统中最可靠的收益来源。后端方向是有前景的，但仍然不成熟；它的失败并非偶然噪声，而是对下一阶段设计有诊断价值的证据。对于课程项目而言，这已经是一个很强的结果：系统不仅展示了“确实存在改善”，还清楚地区分了改善在哪里是稳健的，在哪里是脆弱的，以及这种差异为什么会出现。

8 结论与未来计划

本报告在保持结论与仓库实际状态一致的前提下，将 ME5400 项目重新组织为一个围绕“动态场景下如何提升 LiDAR SLAM 定位精度”的技术叙述。项目真正完成的核心工作，不是单纯把 FAST-LIO、PointPillars 和 MCTrack 拼接成一个系统名称，而是建立了一个可重复运行的对象感知定位增强管线：FAST-LIO 负责主定位，PointPillars 和 MCTrack 提供对象级动态观测，前端动态降权将这些信息直接作用于扫描匹配，ego-only 后端则作为进一步利用对象信息的探索性扩展。配合标准化 ROS 接口、脚本化实

验流程和 7 个 KITTI Tracking 在线序列的归档评估，项目形成了一个能够持续分析定位收益与失效机制的工程验证平台。

当前最重要、也最明确验证过的结论其实很简单。系统中最稳定、最可靠的增益，来自对象感知的前端定位增强链路：FAST-LIO 位姿支撑 MCTrack 形成稳定轨迹，而 MCTrack 输出又反过来为 FAST-LIO 提供动态区域的软抑制信息。这个前端配置在全部 7 个已报告的在线序列上都改善了 ATE RMSE，并在大多数序列上改善了 KITTI 平移误差。因此，当前最成熟、证据最充分的技术结论，应当落在“对象感知前端能够稳定提升动态场景下的定位精度”这一层面，而不是把项目主结论表述为已经完成了完整的联合 SLAMMOT 优化。现有现象强烈暗示，跟踪反馈式动态降权是其中的关键机制，因为它以保守、鲁棒且适合在线运行的方式改善了扫描匹配与局部地图一致性；但在缺少进一步前端消融实验之前，更严谨的写法仍应把它表述为关键机制，而非被独立证明的唯一主因。

本报告同样清楚地说明了哪些问题还没有被解决。当前探索性联合后端并不是一个鲁棒的对象中心化优化器。它只是一个在简化假设下测试可行想法的仅自车滑窗原型。它在漂移主导的序列上有时能起作用，但当对象速度外推、时间对齐或门控条件不够可靠时，也可能出现严重退化。Sequence 0009 就是最关键的提醒：把被跟踪对象当作正向定位约束来复用，要比仅仅把它们用于前端软抑制困难得多。这并不否定项目假设，恰恰相反，它正是将该假设精炼为下一阶段更强设计所需的证据。

因此，未来工作的优先级也很清晰。**第一**，后端应从仅自车修正器演化为一个**对象中心化联合后端**，在其中对象状态与自车位姿一起被优化，而不是只作为外部观测输入。这是最重要的升级，因为它直接回应了当前系统无法在优化内部吸收对象状态误差的问题。**第二**，系统需要**更强的置信度建模与关联策略**。检测得分和轨迹长度是有用的第一步启发，但不足以决定对象因子何时应当强烈影响自车运动。面向置信度的加权、更显式的运动一致性检查以及更丰富的数据关联诊断都仍然需要。**第三**，系统需要**更丰富的目标运动建模**。如果对象状态要从前端辅助信息演化为稳定的后端约束，单一匀速外推显然不够，车辆转向、启停以及不同类型交通参与者的行为差异都需要更合理地表达。**第四**，项目需要在**更多 KITTI Tracking 序列与动态强度更丰富的交通场景**上继续验证，以确认当前“前端稳定、后端脆弱”的模式是否具有更广的场景稳定性。

从课程项目价值的角度看，这是一个强且诚实的结果。该项目已经把多个复杂模块集成进统一工作流，证明了对象级动态信息能够稳定地产生前端定位收益，也明确暴露了通往更强联合后端所面临的真实工程缺口。一个能工作的系统、能在当前仓库工作流内重跑的实验，以及透明的失败分析，正是让当前原型对后续研究依然有价值的原因。如果下一轮迭代能够把今天这种消息级对象反馈提升为真正的对象中心化定位优化图，那么这个项目就能自然地课程规模的工程验证，继续演化为动态场景 LiDAR SLAM 定位增强方向上更实质性的研究贡献。

参考文献

- [1] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “FAST-LIO2: Fast direct LiDAR-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [2] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. CVPR*, 2019, pp. 12697–12705.
- [3] X. Wang, S. Qi, J. Zhao, H. Zhou, S. Zhang, G. Wang, K. Tu, S. Guo, J. Zhao, J. Li, and M. Yang, “MCTrack: A Unified 3D Multi-Object Tracking Framework for Autonomous Driving,” in *Proc. IROS*, 2025.
- [4] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. CVPR*, 2012, pp. 3354–3361.
- [5] R. Scona, M. Jaimez, Y. R. Petillot, M. Fallon, and D. Cremers, “StaticFusion: Background reconstruction for dense RGB-D SLAM in dynamic environments,” in *Proc. ICRA*, 2018.
- [6] C. Yu, Z. Liu, X. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, “DS-SLAM: A semantic visual SLAM towards dynamic environments,” in *Proc. IROS*, 2018.
- [7] B. Bescos, J. M. Facil, J. Civera, and J. Neira, “DynaSLAM: Tracking, mapping and inpainting in dynamic scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, 2018.
- [8] S. Fang and H. Li, “LiDAR SLAMMOT based on Confidence-guided Data Association,” *arXiv preprint arXiv:2412.01041*, 2024.
- [9] Z. Zhu, J. Zhao, K. Huang, X. Tian, J. Lin, and C. Ye, “LIMOT: A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6600–6607, 2024.
- [10] P. Tian and H. Li, “Visual SLAMMOT Considering Multiple Motion Models,” *arXiv preprint arXiv:2411.19134*, 2024.
- [11] J. Morris, Y. Wang, and V. Ila, “Online Dynamic SLAM with Incremental Smoothing and Mapping,” *arXiv preprint arXiv:2509.08197*, 2025.
- [12] X. Tian, Z. Zhu, J. Zhao, G. Tian, and C. Ye, “DL-SLOT: Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Optimization,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1017–1027, 2024.

致谢

当我完成这份 ME5400 项目报告时，我想向所有在这项工作过程中给予我支持的人表达诚挚的感谢。这份报告不仅是对项目本身的总结，也是我在学习期间进行学习、研究与工程实践的一份有意义记录。

首先，我要感谢新加坡国立大学设计与工程学院的所有老师。在硕士学习期间，他们给予了我教学、指导和学术支持。他们的耐心与投入极大加深了我对研究与工程实践的理解。

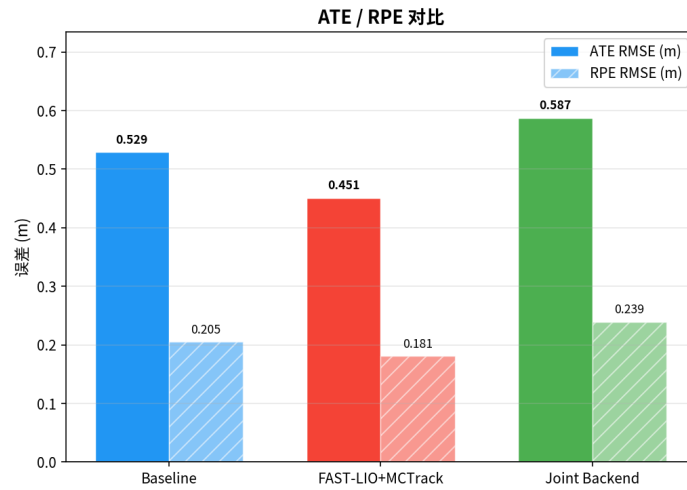
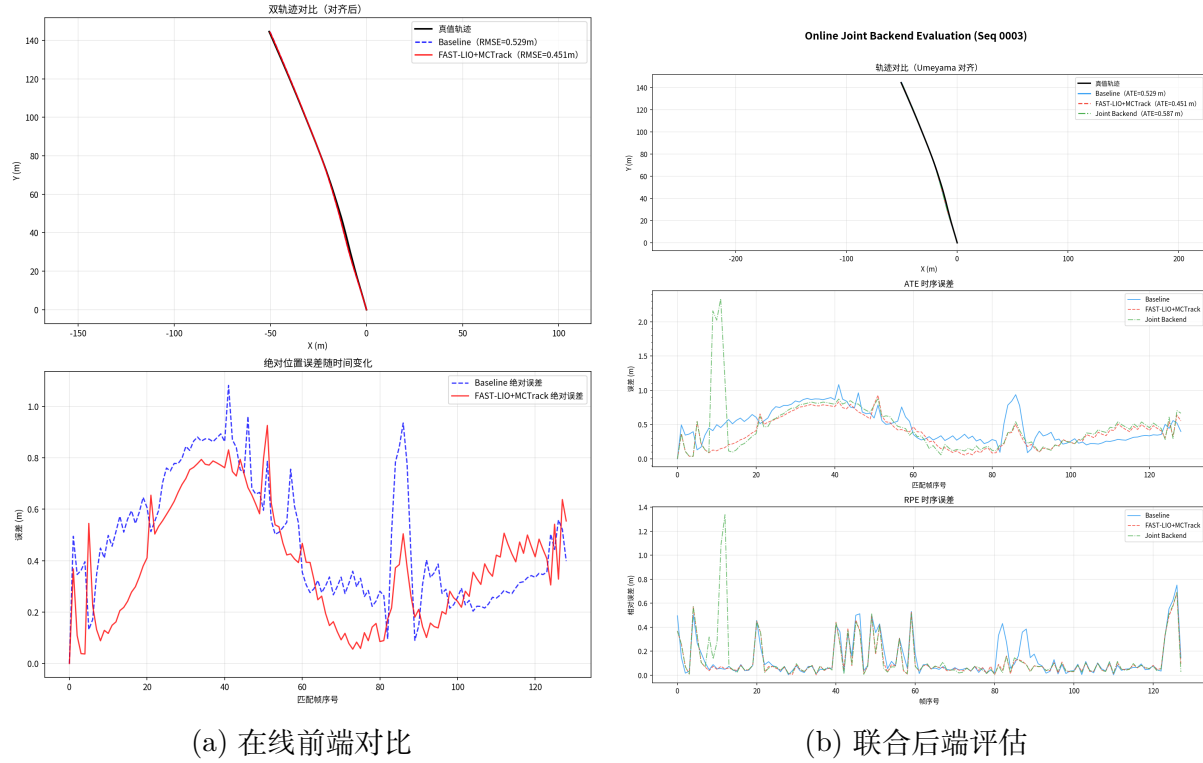
我尤其感谢我的导师 Marcelo H. Ang Jr. 教授，在整个项目过程中给予了我宝贵的指导与支持。我也真诚感谢 Teo Chee Leong 教授，感谢他抽出时间并对本工作进行了细致评估。同时，我也衷心感谢 Han Yuhang，在本研究中给予了我许多有帮助的指导；也感谢研究组中的每一位成员，感谢他们的陪伴、讨论与鼓励。

最后，我要向我的家人和朋友致以最深的敬意和感谢。他们的爱、信任与支持，始终在困难和不确定的时刻给予我力量。

在未来的工作与生活中，我会继续带着这份善意前行，并以真诚、感恩和希望面对接下来的道路。

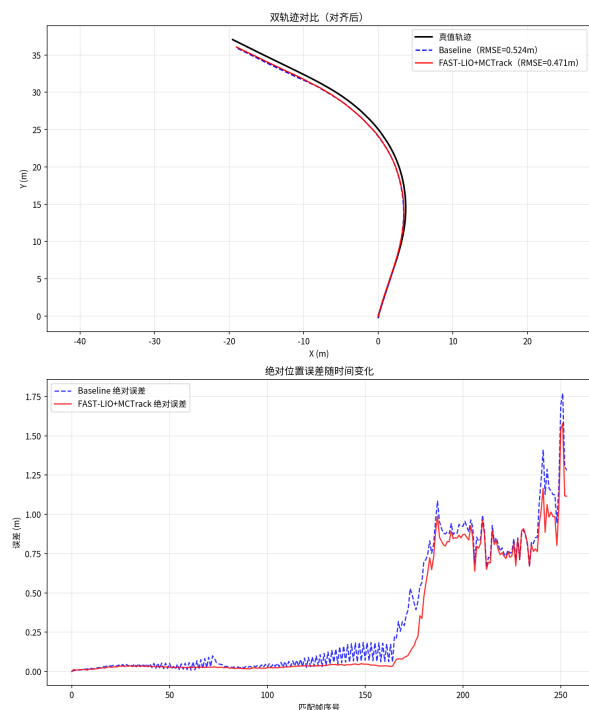
附录：在线实验全量图片

本附录按序列插入全部在线实验结果图。每个序列均包含在线前端对比图、联合后端评估图以及柱状图对比结果。

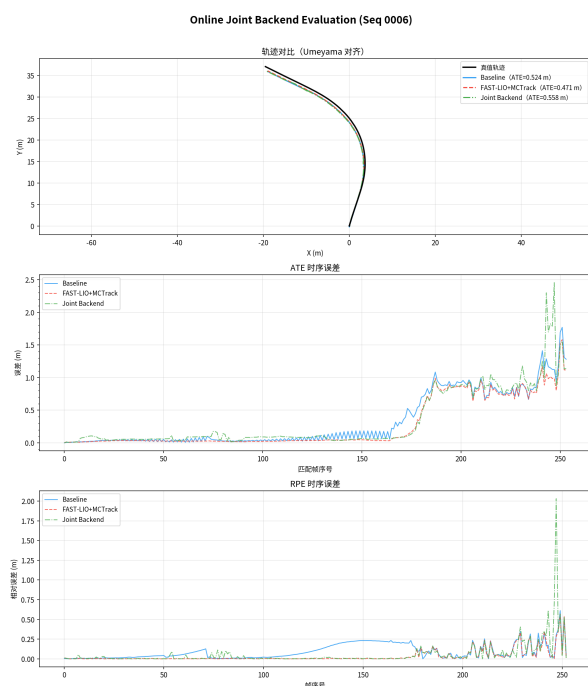


(c) ATE/RPE 柱状对比

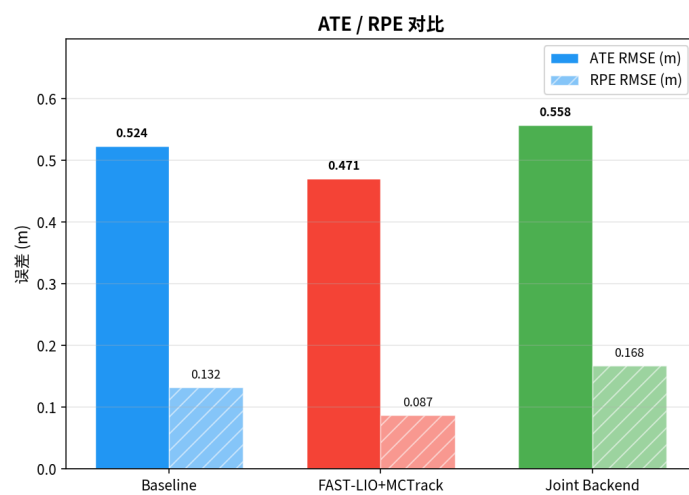
图 9: 序列 0003 的完整在线实验结果。



(a) 在线前端对比

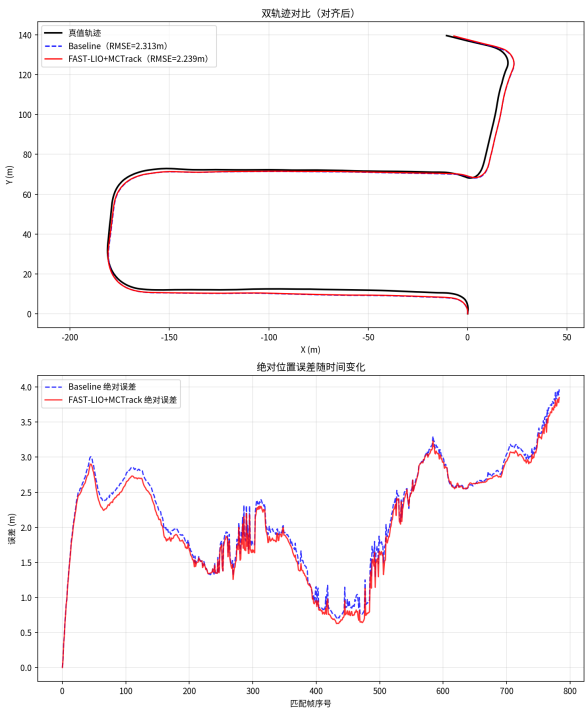


(b) 联合后端评估

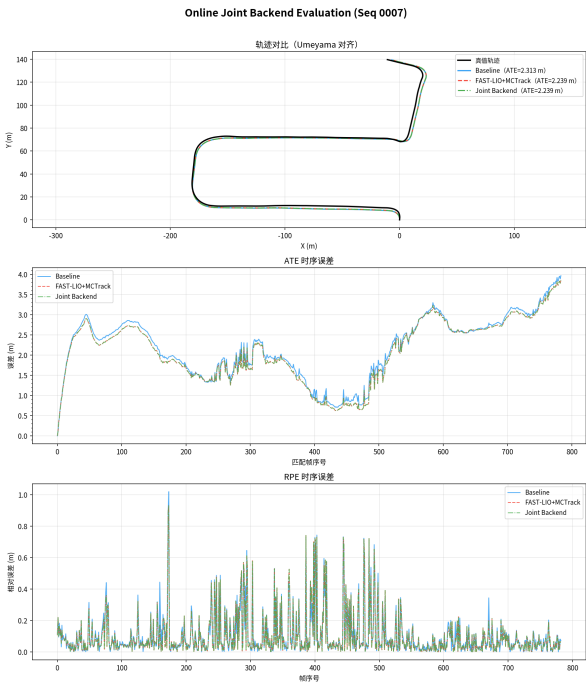


(c) ATE/RPE 柱状对比

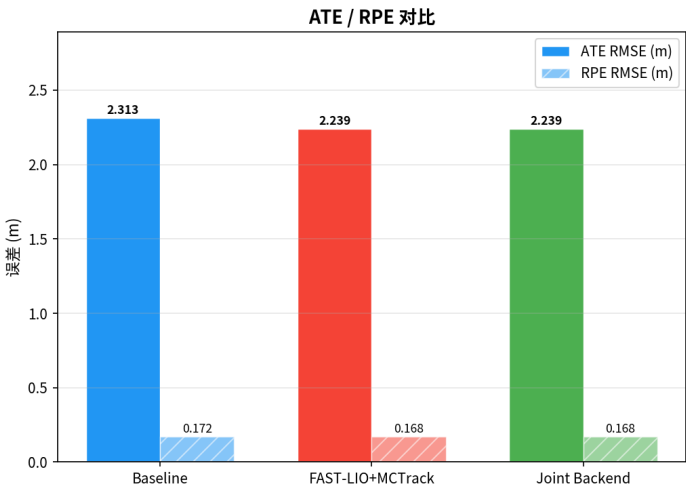
图 10: 序列 0006 的完整在线实验结果。



(a) 在线前端对比

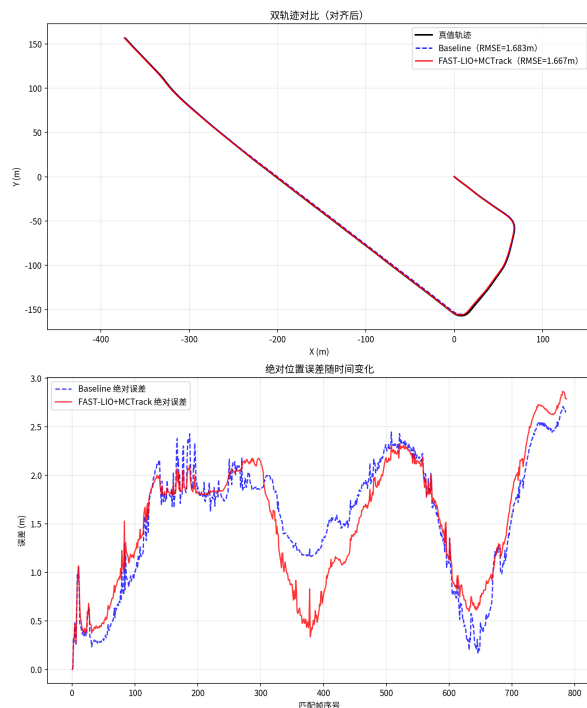


(b) 联合后端评估

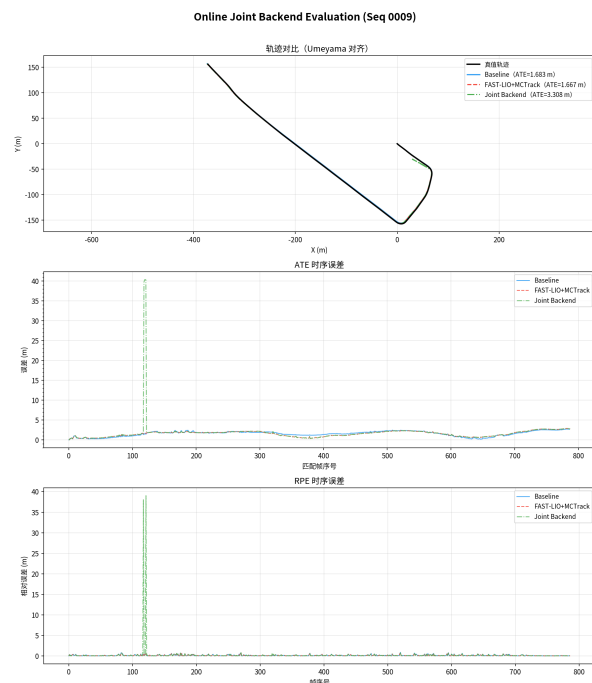


(c) ATE/RPE 柱状对比

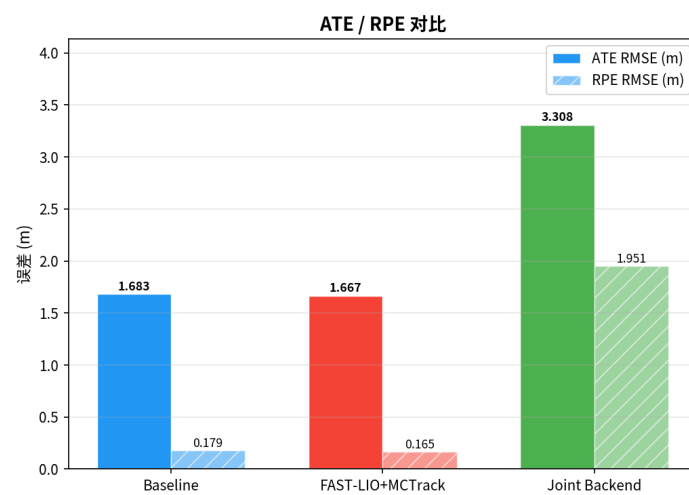
图 11: 序列 0007 的完整在线实验结果。



(a) 在线前端对比

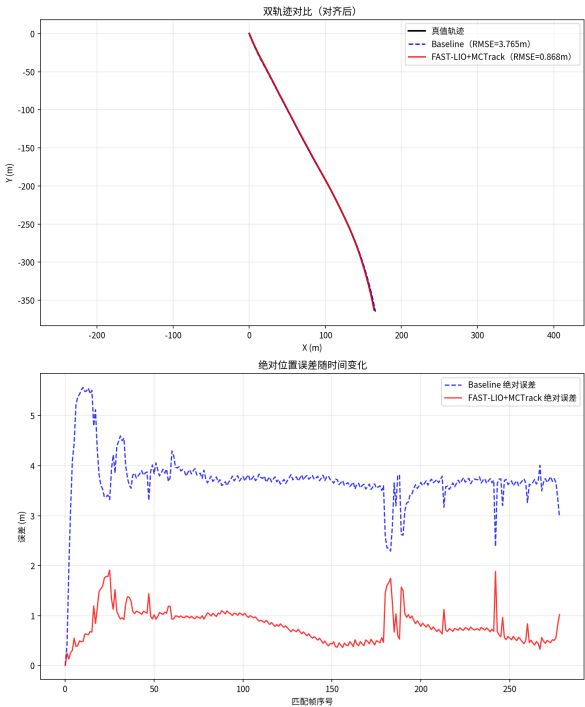


(b) 联合后端评估

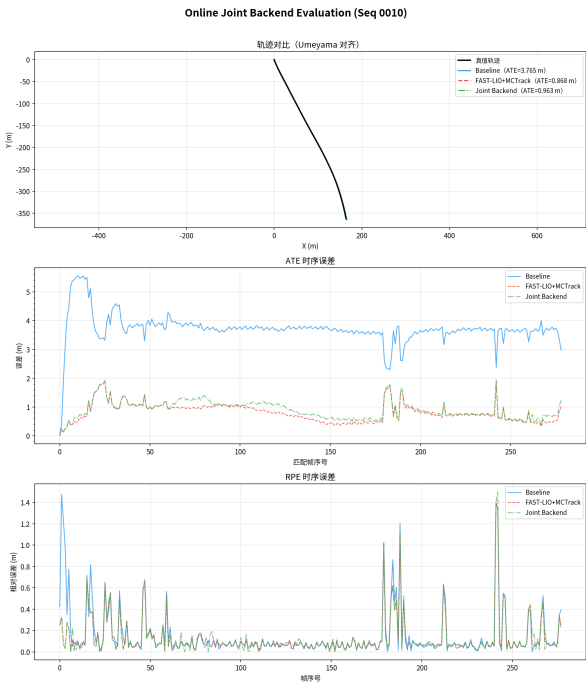


(c) ATE/RPE 柱状对比

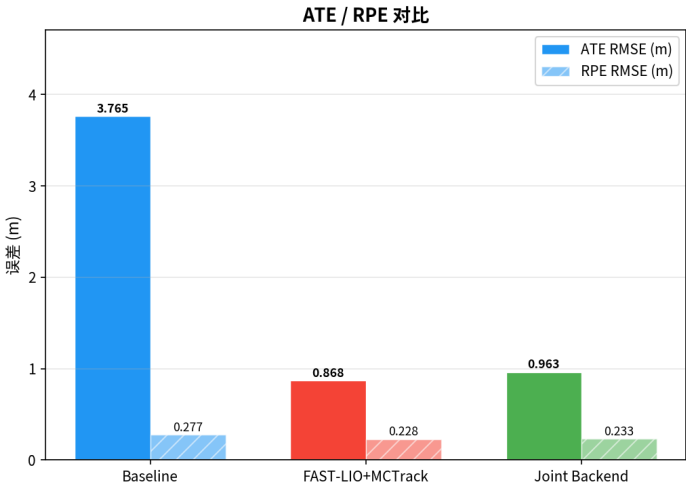
图 12: 序列 0009 的完整在线实验结果。



(a) 在线前端对比

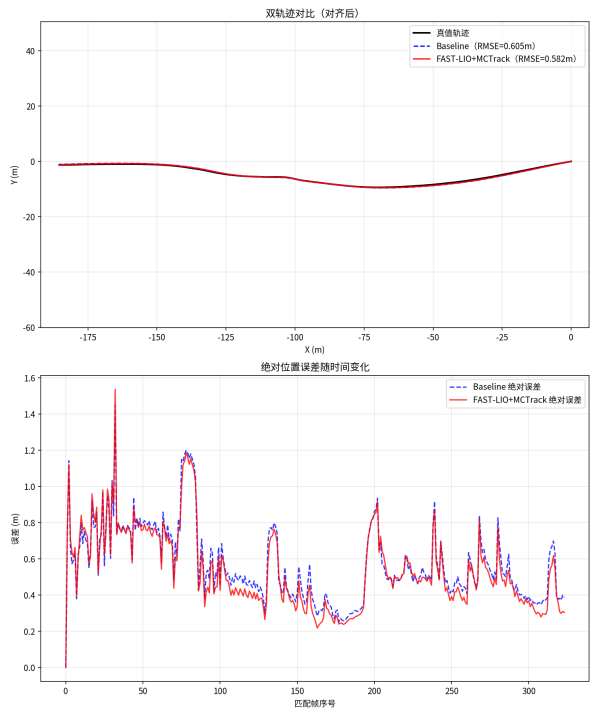


(b) 联合后端评估

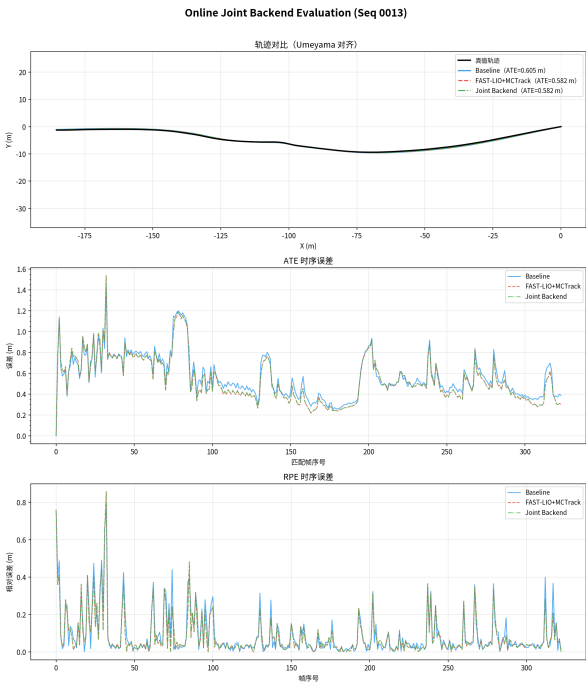


(c) ATE/RPE 柱状对比

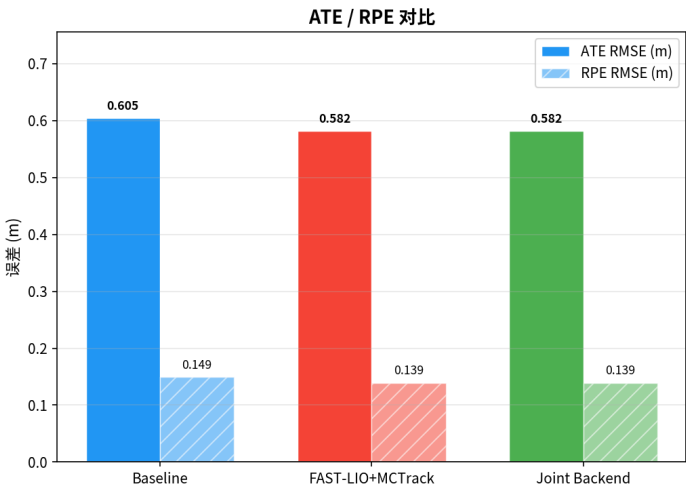
图 13: 序列 0010 的完整在线实验结果。



(a) 在线前端对比

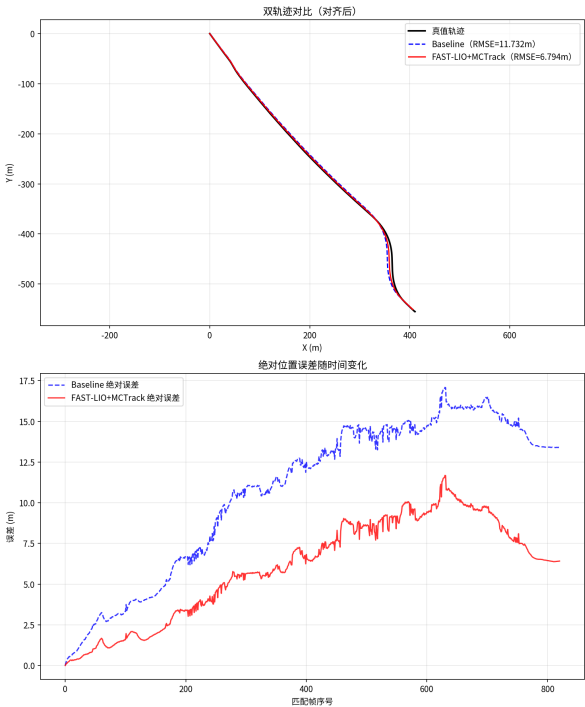


(b) 联合后端评估

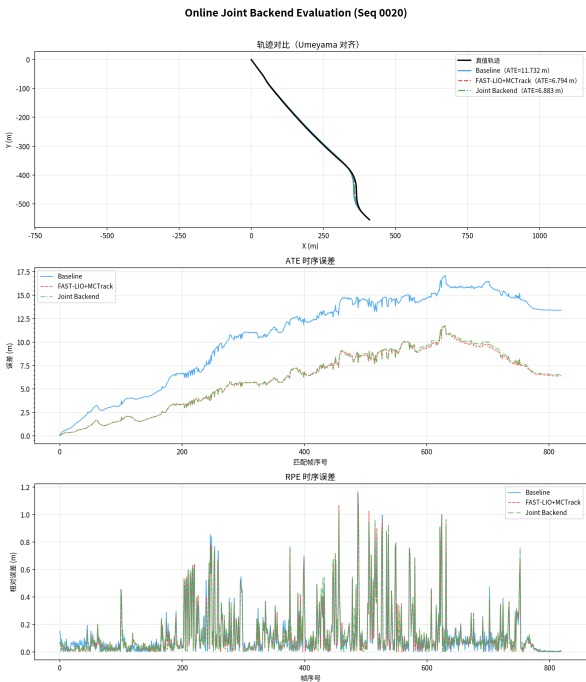


(c) ATE/RPE 柱状对比

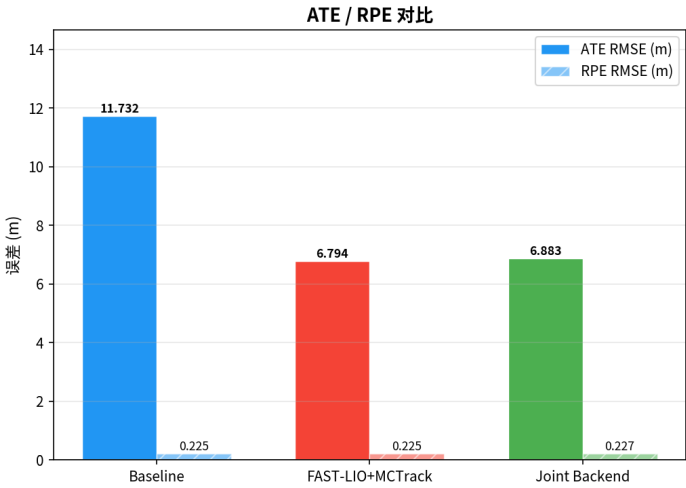
图 14: 序列 0013 的完整在线实验结果。



(a) 在线前端对比



(b) 联合后端评估



(c) ATE/RPE 柱状对比

图 15: 序列 0020 的完整在线实验结果。